

LAPORAN TUGAS BESAR
IF3270 Pembelajaran Mesin
Feedforward Neural Network



Disusun oleh:

Angelica Kierra Ninta Gurning	(13522048)
Kayla Namira Mariadi	(13522050)
Muhammad Neo Cicero Koda	(13522108)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI.....	1
BAB I.....	2
Deskripsi Persoalan.....	2
BAB II.....	3
Pembahasan.....	3
2.1. Penjelasan Implementasi.....	3
2.2. Hasil Pengujian.....	3
2.2.1. Pengaruh depth and width.....	3
2.2.2. Pengaruh fungsi aktivasi.....	3
2.2.3. Pengaruh learning rate.....	4
2.2.4. Pengaruh inisialisasi bobot.....	4
2.2.5. Pengaruh regularisasi.....	5
2.2.6. Pengaruh normalisasi RMSNorm.....	5
2.2.7. Perbandingan dengan library sklearn.....	5
BAB III.....	6
Kesimpulan dan Saran.....	6
3.1. Kesimpulan.....	6
3.2. Saran.....	6
Lampiran.....	7
Referensi.....	8

BAB I

Deskripsi Persoalan

Feedforward Neural Network (FFNN) merupakan bagian dari Artificial Neural Network dalam *machine learning*. ANN merupakan salah satu metode dalam *machine learning* yang meniru cara kerja otak manusia dalam memproses informasi. FFNN digunakan untuk berbagai tugas seperti klasifikasi, regresi, dan pengenalan pola. Model ini terdiri dari lapisan-lapisan neuron yang terhubung secara berurutan, dengan setiap lapisan melakukan transformasi terhadap input menggunakan bobot dan fungsi aktivasi tertentu.

Pada tugas ini, model FFNN akan diimplementasikan dari awal untuk memberikan pemahaman mendalam mengenai bagaimana jaringan saraf bekerja, termasuk mekanisme forward propagation, backward propagation, weight initialization, dan training menggunakan gradient descent.

Terdapat beberapa spesifikasi utama yang harus dipenuhi dalam pengembangan model Feedforward Neural Network, antara lain:

1. **Arsitektur Jaringan** : FFNN harus mendukung jumlah layer dan jumlah neuron per layer yang dapat dikonfigurasi.
2. **Fungsi Aktivasi** : Model harus mendukung fungsi aktivasi Linear, ReLU, Sigmoid, Tanh, dan Softmax.
3. **Fungsi Loss** : Model harus mendukung fungsi loss Mean Squared Error (MSE), Binary Cross-Entropy, dan Categorical Cross-Entropy.
4. **Inisialisasi Bobot** : Bobot dan bias dapat diinisialisasi menggunakan metode Zero Initialization, Uniform Distribution, dan Normal Distribution.
5. **Forward Propagation** : Implementasi perhitungan keluaran jaringan berdasarkan input yang diberikan.
6. **Backward Propagation** : Implementasi perhitungan gradien bobot menggunakan konsep chain rule.
7. **Weight Update** : Pembaruan bobot menggunakan gradient descent.
8. **Visualisasi Model** : Model harus dapat menampilkan struktur jaringan, bobot, dan distribusi gradien dalam bentuk graf.
9. **Penyimpanan dan Pemuatan Model** : Model dapat disimpan dan dimuat kembali untuk penggunaan selanjutnya.

BAB II

Pembahasan

2.1. Penjelasan Implementasi

Pada tugas kali ini, diimplementasikan dua versi yaitu versi tanpa memakai auto diferensial. Untuk dataset MNIST 784 akan diuji menggunakan versi tanpa auto diferensial dikarenakan versi tanpa auto diferensial menggunakan numpy array sehingga lebih efisien. Untuk implementasi auto diferensial, bisa diuji menggunakan dataset yang kecil.

2.1.1. Deskripsi Kelas Untuk Auto Differensial

Tabel 2.1.1.1 Penjelasan Kelas Neuron

Kelas Neuron	
Kelas ini berfungsi untuk mewakili sebuah neuron dengan bobot dan bias	
<code>__init__(self, n_in, weight_init='uniform', weight_params={}, seed=None)</code>	Menginisialisasi neuron dengan n_in input dan metode inisialisasi bobot (zero, uniform, normal, xavier, he)
<code>initialize_weights(self, n_in, weight_init, weight_params, seed)</code>	Mengatur bobot (w) dan bias (b) berdasarkan metode inisialisasi yang dipilih
<code>__call__(self, x)</code>	Mengembalikan net sum dari neuron sesuai dengan input , bobot , dan bias
<code>parameters(self)</code>	Mengembalikan daftar bobot dan bias neuron
<code>get_weights(self)</code>	Mengembalikan nilai bobot dan bias dalam bentuk list
<code>get_grads(self)</code>	Mengembalikan nilai gradien bobot dan bias dalam bentuk list

Tabel 2.1.1.2 Penjelasan Kelas Layer

Kelas Layer	
Kelas ini berfungsi untuk mewakili sebuah layer yang terdiri dari beberapa neuron	
<code>__init__(self, n_in,</code>	Menginisialisasi layer dengan

<code>n_neuron, **kwargs)</code>	<code>n_neuron</code> neuron dan jumlah input <code>n_in</code>
<code>__call__(self, x)</code>	Menghitung keluaran dari semua neuron dalam layer
<code>activate(self, x)</code>	Mengaktifkan output menggunakan fungsi aktivasi yang dipilih
<code>parameters(self)</code>	Mengembalikan daftar parameter dari seluruh neuron di layer ini.
<code>softmax(self, values)</code>	Menghitung fungsi softmax untuk klasifikasi multi-kelas
<code>get_weights(self)</code>	Mengembalikan bobot dan bias semua neuron dalam layer
<code>get_grads(self)</code>	Mengembalikan gradien dari semua neuron dalam layer

Tabel 2.1.1.3 Penjelasan Kelas FFNN

Kelas FFNN	
Kelas ini merupakan modul multi-layer perceptron yang diimplementasikan	
<code>__init__(self, layer_sizes, activations, loss_function='mse', use_rmsnorm=False, **kwargs)</code>	Menginisialisasi jaringan dengan ukuran layer <code>layer_sizes</code> , fungsi aktivasi <code>activations</code> , dan fungsi loss <code>loss_function</code>
<code>__call__(self, x)</code>	Melakukan forward pass melalui semua layer
<code>parameters(self)</code>	Mengembalikan semua parameter yang bisa dilatih dalam jaringa
<code>forward_propagate(self, x)</code>	Melakukan forward propagation untuk satu batch input
<code>backward_propagate(self, loss)</code>	Melakukan backpropagation berdasarkan loss
<code>update_weights(self, lr)</code>	Memperbarui bobot dan bias menggunakan Gradient Descent
<code>get_layer_weights(self, layer_indices):</code>	Mengembalikan bobot dan bias dari layer tertentu

<code>get_layer_grads(self, layer_indices)</code>	Mengembalikan gradien dari layer tertentu
<code>fit(self, x, y, epochs=100, lr=0.01, batch_size=1, verbose=1, regularization_type=None, lambda_reg=0.0)</code>	Melatih jaringan dengan Gradient Descent, termasuk L1/L2 Regularization jika diperlukan.
<code>predict(self, x)</code>	Memprediksi output berdasarkan input x
<code>get_final_weights(self)</code>	Mengembalikan bobot akhir setelah pelatihan
<code>to_json(self)</code>	Menyimpan arsitektur jaringan dan bobot ke dalam format JSON
<code>save(self, path)</code>	Menyimpan model ke file JSON
<code>load(path)</code>	Memuat model dari file JSON dan mengembalikan objek FFNN

Tabel 2.1.1.4 Penjelasan Kelas Values

Kelas Values	
Kelas untuk melakukan diferensiasi secara otomatis	
<code>__init__(self, data, _children=(), _op='')</code>	• <code>data</code>: Menyimpan nilai numerik dari objek Value. • <code>grad</code>: Menyimpan gradien dari nilai tersebut, digunakan dalam backpropagation. • <code>_backward</code>: Fungsi yang digunakan untuk menghitung propagasi gradien saat backpropagation. • <code>_prev</code>: Menyimpan node-node sebelumnya yang menjadi input dalam operasi yang menghasilkan objek ini. • <code>_op</code>: String yang merepresentasikan operasi yang menghasilkan objek ini
<code>__add__(self, other)</code>	Mengembalikan objek Value baru dengan hasil penjumlahan dua angka.

	Saat backpropagation, gradien dari hasil akan diturunkan secara langsung ke kedua operannya
<code>__radd__(self, other)</code>	Mengatasi kasus penjumlahan ketika <code>other + self</code> dipanggil dan <code>other</code> bukan objek <code>Value</code>
<code>__mul__(self, other)</code>	Melakukan perkalian dua objek <code>Value</code> atau bilangan biasa. Gradien dihitung menggunakan aturan rantai diferensial.
<code>__rmul__(self, other)</code>	Mengatasi kasus perkalian <code>other</code> dengan <code>self</code> jika <code>other</code> adalah bilangan biasa
<code>__sub__(self, other)</code>	Mengimplementasikan pengurangan sebagai penjumlahan dengan nilai negatif.
<code>__rsub__(self, other)</code>	Mengatasi kasus pengurangan <code>other</code> dengan <code>self</code> jika <code>other</code> adalah bilangan biasa
<code>__pow__(self, n)</code>	Melakukan perpangkatan dengan eksponen <code>n</code> . Gradien dihitung berdasarkan turunan pangkat.
<code>__truediv__(self, other)</code>	Mengimplementasikan pembagian sebagai perkalian dengan kebalikan <code>other</code>
<code>__rtruediv__(self, other)</code>	Mengatasi kasus pembagian <code>other</code> dengan <code>self</code> jika <code>other</code> adalah bilangan biasa
<code>relu(self)</code>	Fungsi ReLU (Rectified Linear Unit), yang mengembalikan nilai asli jika positif, dan 0 jika negatif. Mengimplementasikan turunan ReLU untuk backward propagation
<code>sigmoid(self)</code>	Fungsi aktivasi sigmoid, serta mengimplementasikan turunan fungsi sigmoid untuk backward propagation
<code>tanh(self)</code>	Fungsi aktivasi tanh, serta mengimplementasikan turunan fungsi tanh untuk backward propagation

<code>leaky_relu(self)</code>	Fungsi aktivasi leaky_relu, serta mengimplementasikan turunan fungsi leaky_relu untuk backward propagation
<code>swish(self)</code>	Fungsi aktivasi swish, serta mengimplementasikan turunan fungsi swish untuk backward propagation
<code>backward_propagate(self)</code>	Melakukan propagasi mundur untuk menghitung gradien semua node yang berkontribusi terhadap nilai tersebut
<code>log(self)</code>	Menghitung logaritma natural dari self.data
<code>abs(self)</code>	Mengembalikan nilai absolut $ x $

Tabel 2.1.1.5 Penjelasan Kelas Visualizer

Kelas Visualizer	
Kelas ini mengimplementasikan visualisasi dari model FFNN	
<code>__init__(self, model)</code>	Menginisialisasi objek Visualizer dengan model neural network
<code>get_network_graph(self)</code>	Membangun struktur graf dengan menambahkan node dan edge
<code>plot_neural_network(self)</code>	Menghasilkan visualisasi jaringan saraf menggunakan Matplotlib dan NetworkX

Tabel 2.1.1.6 Penjelasan Kelas RMSNorm

Kelas RMSNorm	
Kelas ini mengimplementasikan Root Mean Square Normalization (RMSNorm) yang menormalkan input dengan menghitung rata-rata kuadrat elemen-elemen dalam input	
<code>__init__(self, dim, epsilon=1e-5)</code>	Menginisialisasi normalisasi RMS dengan parameter dim (jumlah elemen yang akan dinormalisasi), epsilon digunakan untuk mencegah pembagian dengan nol, gamma adalah parameter skala untuk setiap elemen setelah

	normalisasi.
<code>__call__(self, x)</code>	Menghitung rata-rata kuadrat dari elemen x, menormalkan nilai dengan $\sqrt{\text{mean_sq} + \text{epsilon}}$, mengalikan hasil dengan gamma untuk mempertahankan skala
<code>parameters(self)</code>	Mengembalikan parameter gamma

2.1.2. Penjelasan Forward Propagation

Forward propagation merupakan proses data input diteruskan dari input layer hingga output layer untuk menghasilkan output layer. Input bisa melalui berbagai layer dengan jumlah tertentu. Dari input layer, masukkan akan diteruskan menuju hidden layer dan pada akhirnya output layer. Pada tiap neuron pada *layer* akan dilakukan operasi penjumlahan dengan bobot dan bias, dan kemudian hasilnya akan diolah menggunakan fungsi aktivasi. Operasi penjumlahan pada tiap neuron dilakukan sebagai berikut:

$$z = \sum_{i=1}^n (w_i \cdot x_i) + b$$

di mana:

- x_i adalah nilai input dari neuron sebelumnya
- w_i adalah bobot yang dikalikan dengan masing-masing input
- b adalah bias
- z adalah nilai sebelum fungsi aktivasi diterapkan

Setelah mendapatkan nilai z , nilai akan dimasukkan ke fungsi aktivasi. Pada implementasi ini terdapat fungsi aktivasi ReLu, Sigmoid, Linear, Softmax, Swish, dan Leaky ReLu.

2.1.2.1. Penjelasan Alur Implementasi Forward Propagation

Pada implementasi tugas ini, FFNN menggunakan kelas Value untuk operasi-operasi dasar dan sebagai penyimpang objek perhitungan. Setiap objek `value` :

- Menyimpan nilai numerik (`data`)

- Menyimpan gradien (*grad*)
- Menyimpan informasi tentang bagaimana nilai dihitung (*_op* dan *_prev*)
- Menyimpan fungsi *_backward* untuk menghitung turunan lokal saat backward propagation

Sebagai contoh, forward propagation menyusun objek *Value* dan mencatat graf komputasionalnya

$$z = w * x + b$$

Operasi ini tidak hanya menghitung nilai z , tapi juga menyimpan jejak bahwa z berasal dari w , x , dan b . Graf ini akan digunakan saat backward propagation.

Berikut merupakan proses Forward Propagation di model FFNN yang dibuat:

1. Input diterima oleh model (FFNN)
2. Input diteruskan ke setiap layer (Layer)
3. Di dalam layer, input diproses oleh neuron-neuron (Neuron)
4. Masing-masing neuron menghitung weighted sum (dot product) + bias
5. Output neuron melewati fungsi aktivasi (dan RMSNorm jika diaktifkan)
6. Output akhir dari jaringan digunakan untuk menghitung loss

2.1.2.2. Penjelasan Kode Implementasi

Kode & Penjelasan

```
1 def __call__(self, x):
2     for i in range(1, len(self.layers)):
3         x = self.layers[i](x)
4     return x
```

Kelas FFNN

- Forward propagation berlangsung melalui semua lapisan dalam jaringan, mulai dari input hingga output.
- Output dari satu lapisan menjadi input untuk lapisan berikutnya hingga mencapai lapisan terakhir.

```

1  def __call__(self, x):
2      raw_out = [neuron(x) for neuron in self.neurons]
3
4      if self.use_rmsnorm:
5          raw_out = self.norm(raw_out)
6      if self.activation == "softmax":
7          softmaxed = self.softmax(raw_out)
8          return softmaxed
9
10     return [self.activate(o) for o in raw_out] if len(raw_out) > 1 else self.activate(raw_out[0])

```

Kelas Layer

- Setiap neuron dalam lapisan menerima input dan menghasilkan output yang kemudian diteruskan ke fungsi aktivasi.
- Forward propagation dalam layer terjadi dengan memanggil setiap neuron satu per satu, lalu menerapkan fungsi aktivasi.

```

1  def __call__(self, x):
2      return sum((wi*xi for wi,xi in zip(self.w, x)), self.b)

```

Kelas Neuron

- Operasi untuk menghitung keluaran neuron berdasarkan input dan bobotnya.

2.1.3. Penjelasan Backward Propagation

Backward Propagation merupakan metode untuk menghitung gradien dari fungsi loss terhadap bobot jaringan menggunakan *chain rule* dari turunan/diferensial. Jika L adalah fungsi loss, dan w adalah bobot yang diperbarui, maka gradien akan dihitung sebagai berikut:

$$\frac{dL}{dw} = \frac{dL}{dz} \cdot \frac{dz}{dw}$$

2.1.3.1. Penjelasan Alur Implementasi Backward Propagation

Berikut merupakan proses implementasi Backward Propagation pada tugas ini, auto diferensial dilakukan pada kelas `Value` :

- Setelah prediksi diperoleh, fungsi loss digunakan untuk menghitung error.
- Fungsi `backward_propagate()` pada objek loss memicu pelacakan backward melalui computational graph.
- Fungsi ini menelusuri node secara topological order dari output ke input (bottom-up).
- Setiap node memanggil fungsi `_backward()` yang disimpan saat forward untuk menyebarkan turunan ke node sebelumnya.

2.2. Deskripsi Kelas untuk Non-Auto Diferensial

Fungsi Aktivasi	
<code>relu(x)</code>	Mengimplementasikan fungsi aktivasi Rectified Linear Unit (ReLU)
<code>d_relu(x)</code>	Turunan fungsi ReLU, berguna saat backpropagation
<code>sigmoid(x)</code>	Fungsi aktivasi sigmoid untuk klasifikasi biner
<code>d_sigmoid(x)</code>	Turunan sigmoid
<code>tanh(x)</code>	Fungsi aktivasi tanh (tangent hyperbolic)
<code>d_tanh(x)</code>	Turunan tanh
<code>linear(x)</code>	Fungsi aktivasi linear
<code>d_linear(x)</code>	Turunan linear
<code>softmax(x)</code>	Fungsi aktivasi softmax untuk klasifikasi multi-class.
<code>d_softmax_crossentropy(y_p</code>	Turunan gabungan softmax dan

<code>red, y_true)</code>	cross-entropy.
---------------------------	----------------

Fungsi Loss	
<code>mse(y_pred, y_true)</code>	Menghitung mean squared error (MSE)
<code>d_mse(y_pred, y_true)</code>	Turunan MSE untuk backpropagation
<code>bce(y_pred, y_true)</code>	Binary Cross Entropy, untuk klasifikasi biner
<code>d_bce(y_pred, y_true)</code>	Turunan binary cross entropy
<code>cce(y_pred, y_true)</code>	Categorical Cross Entropy, untuk klasifikasi multi-class

Kelas Layer	
Kelas ini berfungsi untuk mewakili sebuah layer yang terdiri dari beberapa neuron	
<code>__init__(self, n_in, n_out, activation='linear', weight_init='xavier', use_rmsnorm=False)</code>	Menginisialisasi layer dengan <code>n_neuron</code> neuron dan jumlah input <code>n_in</code>
<code>forward(x)</code>	• Menghitung forward propagation ($z = Wx + b$). • Mengaplikasikan RMSNorm (opsional). • Mengaplikasikan fungsi aktivasi.
<code>backward(grad_output, y_true=None)</code>	Melakukan backward propagation, menghitung gradient untuk bobot dan bias berdasarkan gradien error dari lapisan selanjutnya
<code>rmsnorm(x, epsilon)</code>	Menormalisasi input menggunakan RMS Normalization

Kelas FFNN	
Kelas ini merupakan modul multi-layer perceptron yang diimplementasikan	
<code>__init__(self, layer_sizes, activations, loss_function='mse', weight_init='xavier', use_rmsnorm=False, reg_type=None)</code>	Menginisialisasi jaringan dengan ukuran layer <code>layer_sizes</code> , fungsi aktivasi <code>activations</code> , dan fungsi loss <code>loss_function</code>
<code>forward(x)</code>	Menjalankan forward propagation dari input hingga output melalui semua lapisan
<code>update_weights(grads, lr, reg_type, lambda_reg)</code>	Meng-update bobot dan bias berdasarkan gradien, learning rate, dan regularisasi (opsional)
<code>fit(x, y, epochs, lr, batch_size, verbose, lambda_reg)</code>	• Melatih neural network menggunakan metode mini-batch gradient descent. • Menggunakan tqdm untuk menampilkan progress training secara visual. • Menyimpan riwayat training (loss per epoch)
<code>predict(x)</code>	• Menghasilkan prediksi berdasarkan input setelah training. • Mengembalikan klasifikasi atau regresi sesuai fungsi loss yang dipilih
<code>get_weights()</code>	Mengambil bobot dan bias saat ini dari setiap lapisan
<code>set_weights(weights)</code>	Mengatur bobot dan bias neural network
<code>to_json()</code>	Mengonversi konfigurasi training dan bobot akhir menjadi format JSON agar dapat disimpan
<code>save(path)</code>	Menyimpan konfigurasi jaringan serta bobot akhir ke dalam file JSON di lokasi yang diberikan

<code>load(path)</code>	Memuat neural network dari file JSON yang tersimpan
-------------------------	---

2.2.1. Penjelasan Forward Propagation

Untuk implementasi forward propagation mirip seperti dengan auto-diferensial , dimana proses matematisnya sebagai berikut :

$$Z = XW^T + b$$

$$A = f(Z)$$

di mana:

- X adalah nilai input dari neuron sebelumnya
- W adalah bobot yang dikalikan dengan masing-masing input
- b adalah bias
- Z adalah nilai sebelum fungsi aktivasi diterapkan
- A adalah hasil aktivasi

Kode

```

1 def forward(self, x):
2     z = x @ self.W.T + self.b
3     if self.use_rmsnorm:
4         z = self.rmsnorm(z)
5     a = self.act_fn(z) if self.activation_name != 'softmax' else softmax(z)
6     self.cache = {'x': x, 'z': z, 'a': a}
7     return a

```

Kelas Layer

- Mengalikan input dengan bobot (W) dan menambahkan bias (b) untuk mendapatkan nilai aktivasi awal zzz.
- Jika RMSNorm diaktifkan, zzz akan dinormalisasi.
- Hasil zzz kemudian diteruskan ke fungsi aktivasi untuk menghasilkan output aaa.
- Output ini akan menjadi input untuk layer berikutnya.



```
1  def forward(self, x):  
2      for layer in self.layers:  
3          x = layer.forward(x)  
4      return x  
5
```

Kelas FFNN

- Forward propagation pada kelas **FFNN** dilakukan dengan meneruskan input secara berurutan melalui setiap layer yang ada di dalam jaringan. Setiap layer akan menerima input dari layer sebelumnya, lalu menghitung output menggunakan bobot, bias, dan fungsi aktivasi.
- Proses ini dilakukan dengan memanggil fungsi **forward()** pada setiap layer secara berurutan. Output akhir dari layer terakhir adalah hasil prediksi model.

2.2.2. Penjelasan Backward Propagation

Untuk implementasi Backward Propagation akan dihitung turunan dari *loss* terhadap bobot dan bias menggunakan *chain rule*, dengan tahapan sebagai berikut:

dL/dA = turunan dari fungsi loss terhadap output layer (dari output paling akhir)

dA/dZ = turunan dari fungsi aktivasi terhadap Z

dZ/dW = X (input dari layer sebelumnya)

dZ/db = 1

dZ/dX = W

Kemudian gradien dihitung dengan rumus :

$dL/dZ = dL/dA * dA/dZ$	
$dL/dW = (dL/dZ)^T \cdot X / N$	: gradien terhadap bobot
$dL/db = \text{mean}(dL/dZ)$	: gradien terhadap bias
$dL/dX = dL/dZ \cdot W$	: diteruskan ke layer sebelumnya

Di mana:

- X adalah input dari layer sebelumnya (disimpan saat forward)
- Z adalah output sebelum fungsi aktivasi
- A adalah hasil aktivasi
- N adalah jumlah data dalam satu batch
- dL/dX dikembalikan dan digunakan untuk proses backward di layer sebelumnya

Selanjutnya gradien digunakan di `update_weights()` untuk mengoptimasi bobot dan menyesuaikan penurunan *loss*

Kode

```

1  def backward(self, grad_output, y_true=None):
2      x = self.cache['x']
3      z = self.cache['z']
4      a = self.cache['a']
5
6      if self.activation_name == 'softmax':
7          dz = d_softmax_crossentropy(a, y_true)
8      else:
9          dz = grad_output * self.d_act_fn(z)
10
11
12      dW = dz.T @ x / x.shape[0]
13      db = np.mean(dz, axis=0)
14      dx = dz @ self.W
15      self.G = dW
16
17      return dx, dW, db

```

Kelas Layer

- dz adalah turunan dari loss terhadap pre-activation output.
- dW dihitung sebagai rata-rata outer product antara dz dan input.
- db adalah rata-rata dari dz di seluruh batch.
- dx dibutuhkan untuk menghitung turunan ke layer sebelumnya.

```
1 def backward(self, y_pred, y_true):
2     grads = []
3     grad = self.d_loss_fn(y_pred, y_true) if self.d_loss_fn else None
4
5     for i in reversed(range(len(self.layers))):
6         if self.layers[i].activation_name == 'softmax':
7             grad, dW, db = self.layers[i].backward(grad_output=None, y_true=y_true)
8         else:
9             grad, dW, db = self.layers[i].backward(grad)
10        grads.append((dW, db))
11    return grads[::-1]
```

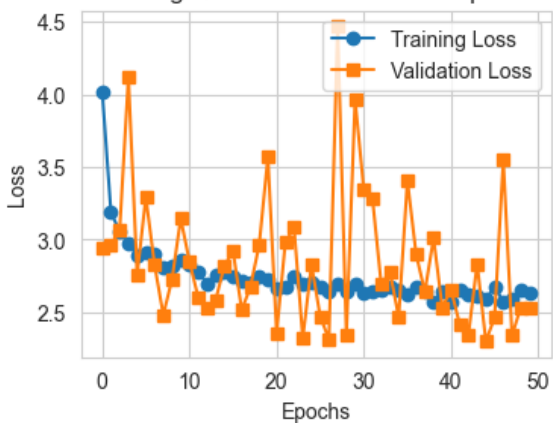
Kelas FFNN

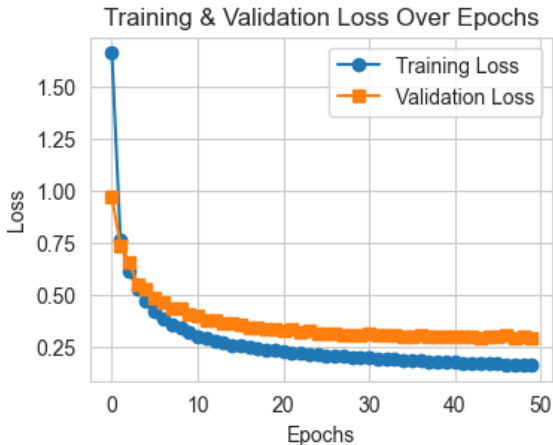
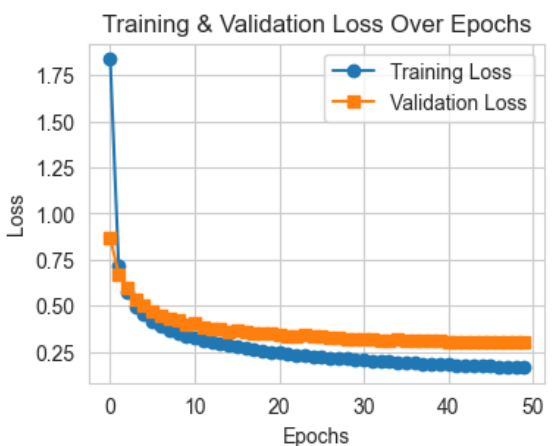
- Gradien pertama dihitung dari turunan loss terhadap output prediksi.
- Kemudian, setiap layer menghitung gradiennya (dW dan db), dan meneruskan gradien terhadap input ke layer sebelumnya.
- Hasil akhirnya adalah daftar gradien untuk semua layer, siap digunakan untuk update bobot.

2.3. Hasil Pengujian

2.2.1. Pengaruh depth and width

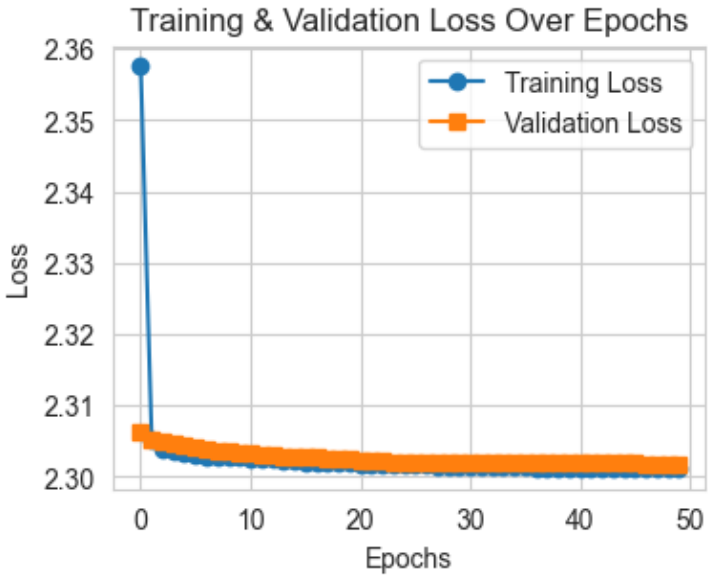
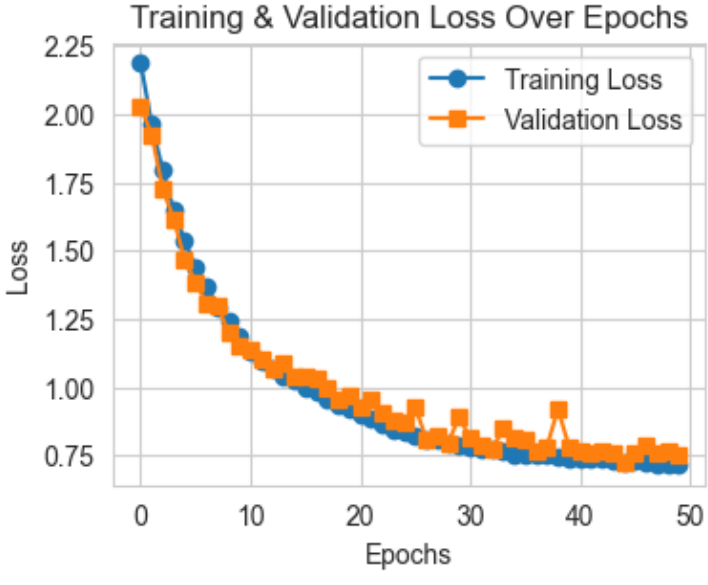
Tabel 2.2.1.1 Hasil Pengujian terhadap pengaruh depth (width tetap)

Depth	Hasil
Tanpa hidden layer, ([784, 10])	Training & Validation Loss Over Epochs  Akurasi: Val: 89,30%, Test: 91.24%

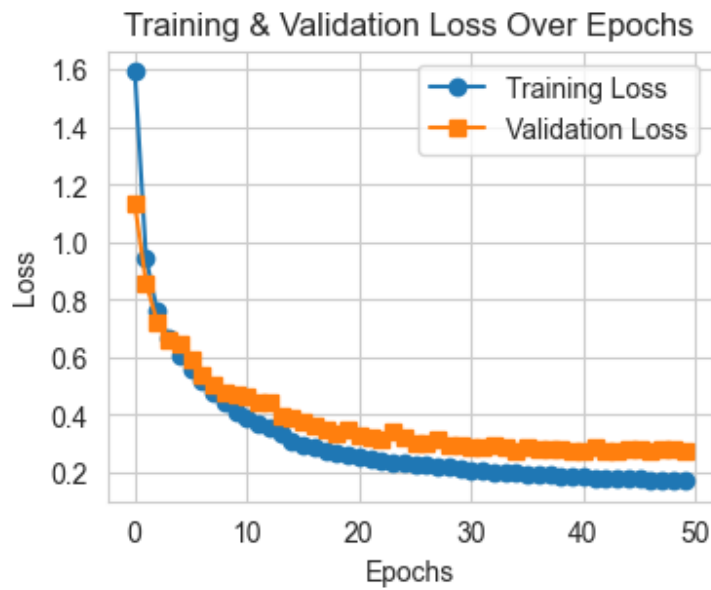
1 hidden layer ([784, 64, 10])	 Akurasi: Val: 93,28%, Test: 95,28%
2 hidden layer ([784, 64, 64, 10])	 Akurasi: Val: 92,73%, Test: 94,87%

Tabel 2.2.1.2 Hasil Pengujian terhadap pengaruh width (depth tetap)

Width	Hasil
-------	-------

1 hidden layer, 3 neuron	Training & Validation Loss Over Epochs  Akurasi: Val: 11,13%, Test: 11,36%
1 hidden layer, 24 neuron	Training & Validation Loss Over Epochs  Akurasi: Val: 74.51%, Test: 75,65%

1 hidden layer,
64 neuron



Akurasi: Val: 93,22%, Test: 95,11%

a. Perbandingan hasil akhir prediksi

Dapat terlihat bahwa model yang memiliki hidden layer memiliki kinerja yang lebih baik daripada model yang tidak memiliki hidden layer. Hal ini menunjukkan bahwa penambahan hidden layer pada neural network memiliki dampak positif. Hidden layer yang digabungkan dengan fungsi aktivasi dapat membantu neural network dalam mempelajari pola nonlinear. Selain itu, hidden layer dapat membantu neural network dalam membuat representasi fitur yang lebih berguna dalam melakukan prediksi. Hidden layer juga dapat membantu model dalam melakukan generalisasi berdasarkan fitur-fitur yang diberikan. Meskipun itu, dapat terlihat bahwa model yang memiliki dua hidden layer dengan jumlah neuron pada kedua layer yang sama tidak memiliki nilai akurasi yang lebih besar daripada model dengan satu hidden layer. Hal tersebut mungkin terjadi karena kedua hidden layer pada model tersebut memiliki jumlah neuron yang sama sehingga informasi yang dipelajari oleh model mungkin bersifat redundan dalam kedua layer.

Selain itu, dari eksperimen tersebut terlihat bahwa model yang memiliki width lebih lebar memiliki hasil prediksi yang lebih baik. Hal tersebut mungkin terjadi karena model pertama hanya memiliki tiga neuron

pada hidden layer sehingga informasi yang bisa diambil dari layer sebelumnya kurang tertangkap. Hasil prediksi mungkin dapat lebih baik seiring dengan bertambahnya width hidden layer karena layer tersebut memiliki tempat lebih untuk mendapat informasi dalam layer sebelumnya. Akan tetapi, semakin lebar suatu layer, semakin banyak parameter yang ada sehingga waktu yang diperlukan untuk melatih model akan semakin lama.

b. Perbandingan grafik loss pelatihan

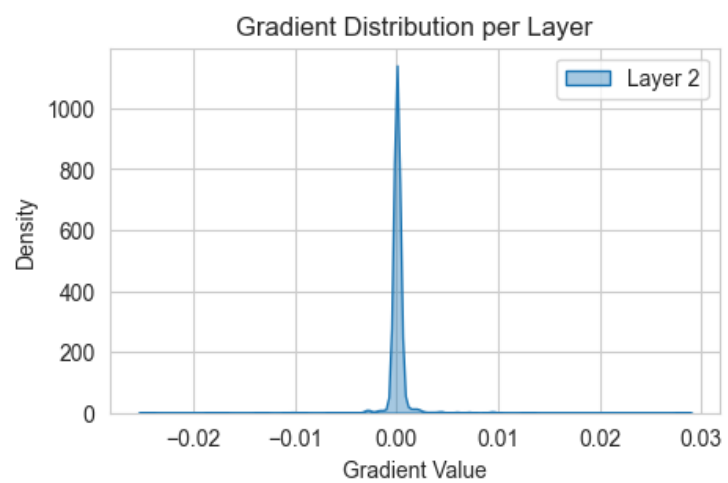
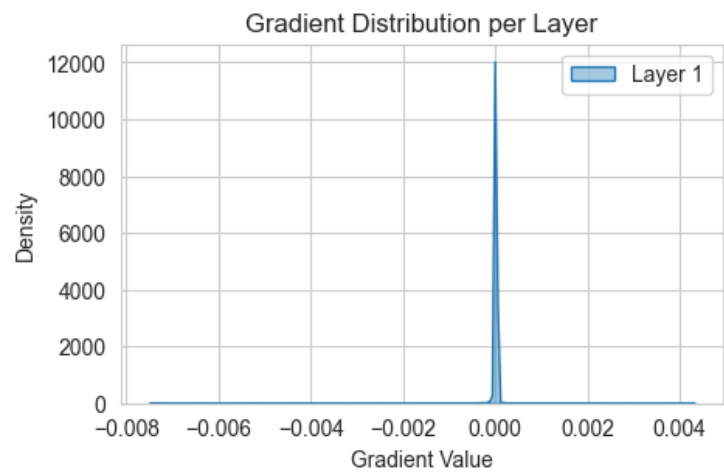
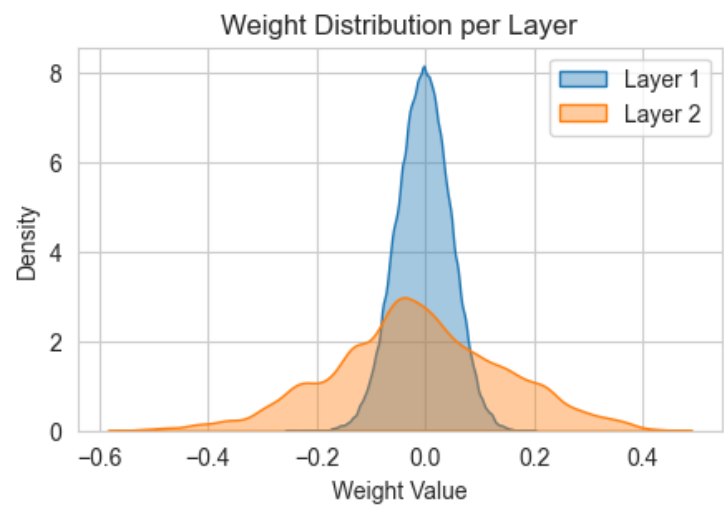
Pada perbandingan model berdasarkan depth, terlihat bahwa model tanpa hidden layer memiliki nilai training loss yang menurun, lalu stagnan dan tidak mencapai angka di bawah 2,5. Terlihat juga bahwa validation loss pada model tersebut sangat fluktuatif. Kedua model yang memiliki hidden layer menghasilkan grafik loss yang terlihat mirip. Kemiripan tersebut mungkin terjadi karena menambahkan hidden layer baru dengan jumlah neuron yang sama dengan hidden layer sebelumnya tidak terlalu mempengaruhi kinerja model.

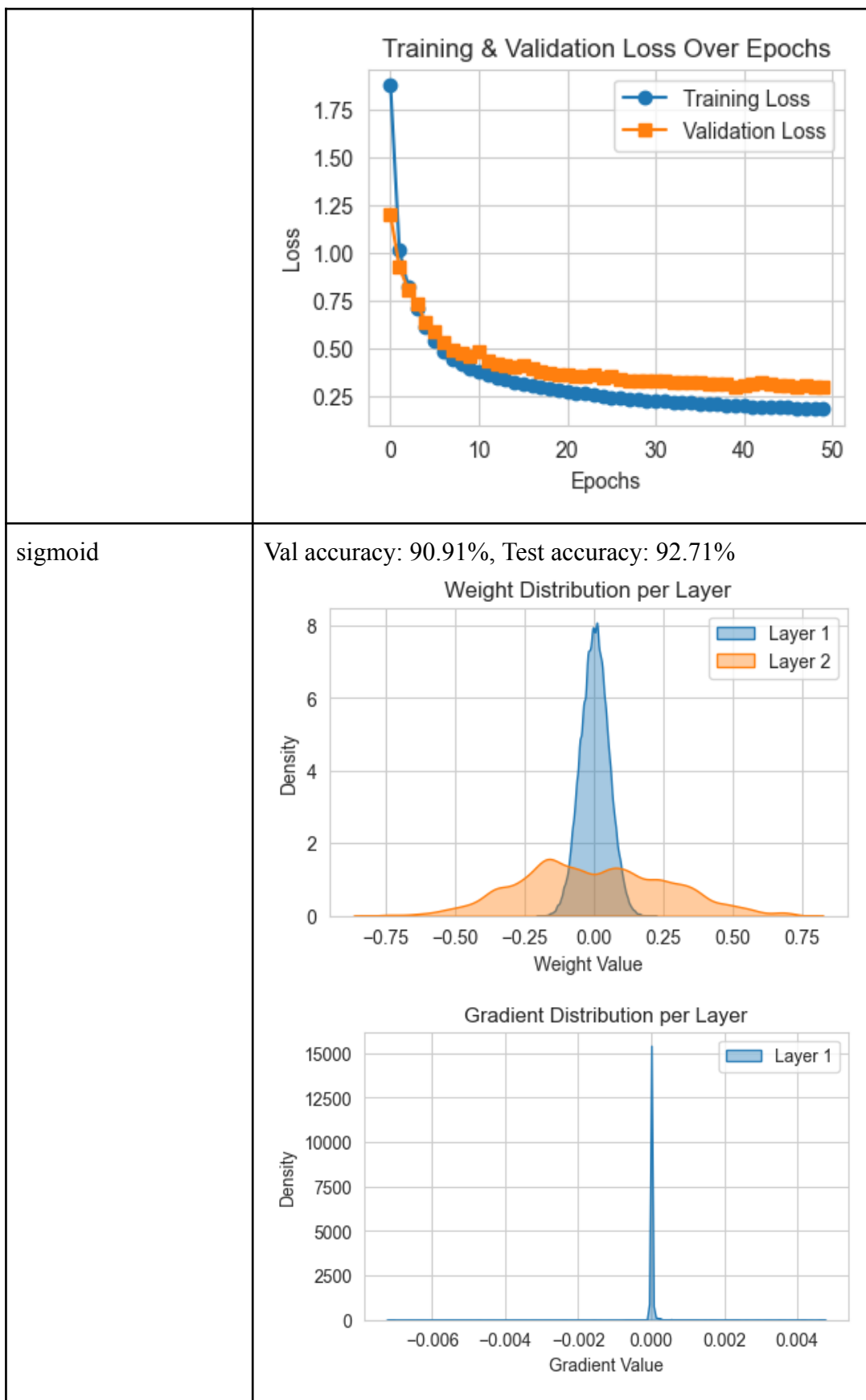
Pada perbandingan model berdasarkan width, dapat terlihat bahwa model dengan 3 neuron pada hidden layer langsung mengalami loss yang stagnan setelah epoch awal. Model dengan 24 neuron memiliki kinerja yang lebih baik, tetapi model dengan 64 neuron memiliki kinerja yang lebih baik. Hal tersebut menunjukkan bahwa lebar suatu layer dapat mempengaruhi neural network dalam mempelajari pola yang baik.

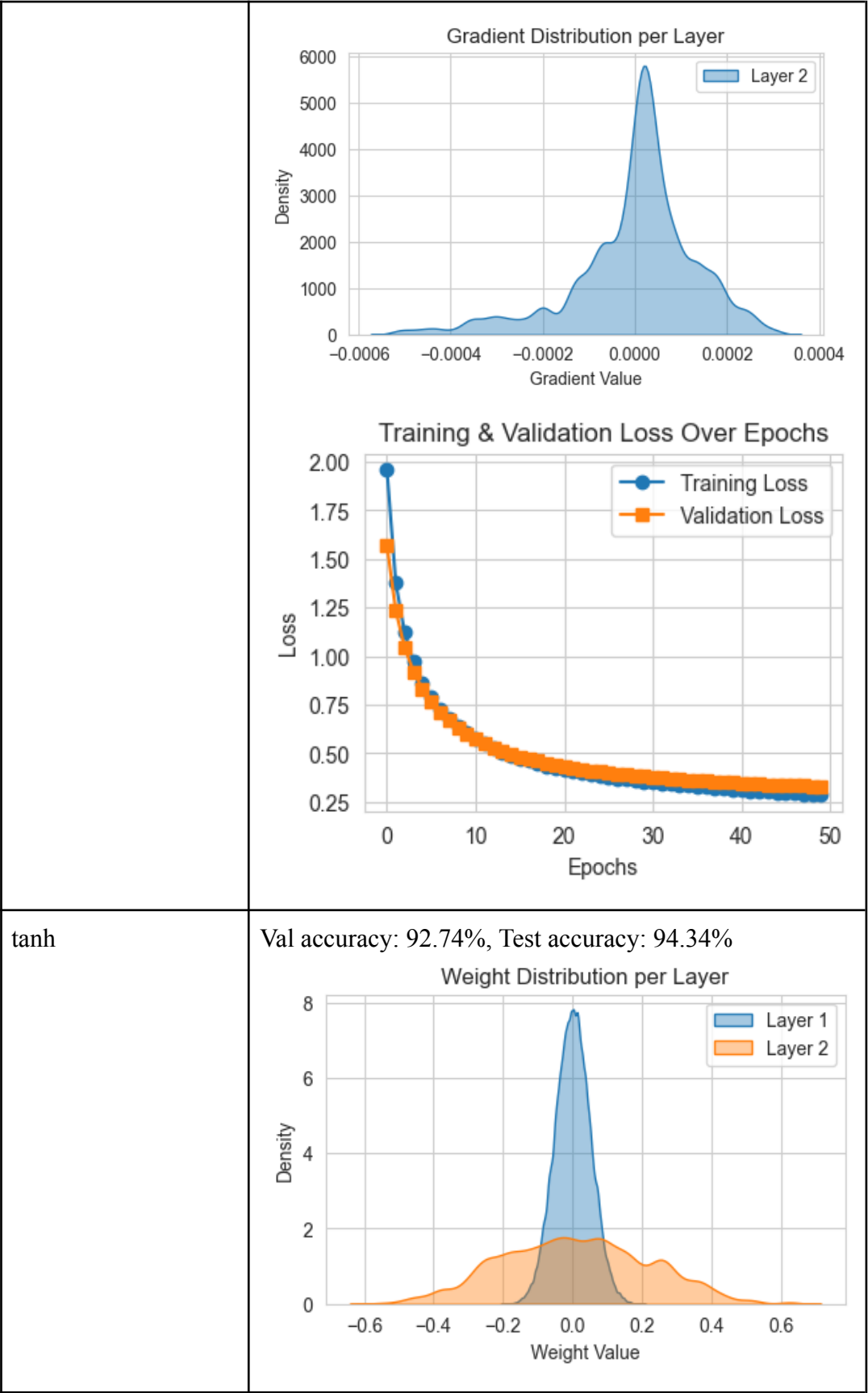
2.2.2. Pengaruh fungsi aktivasi

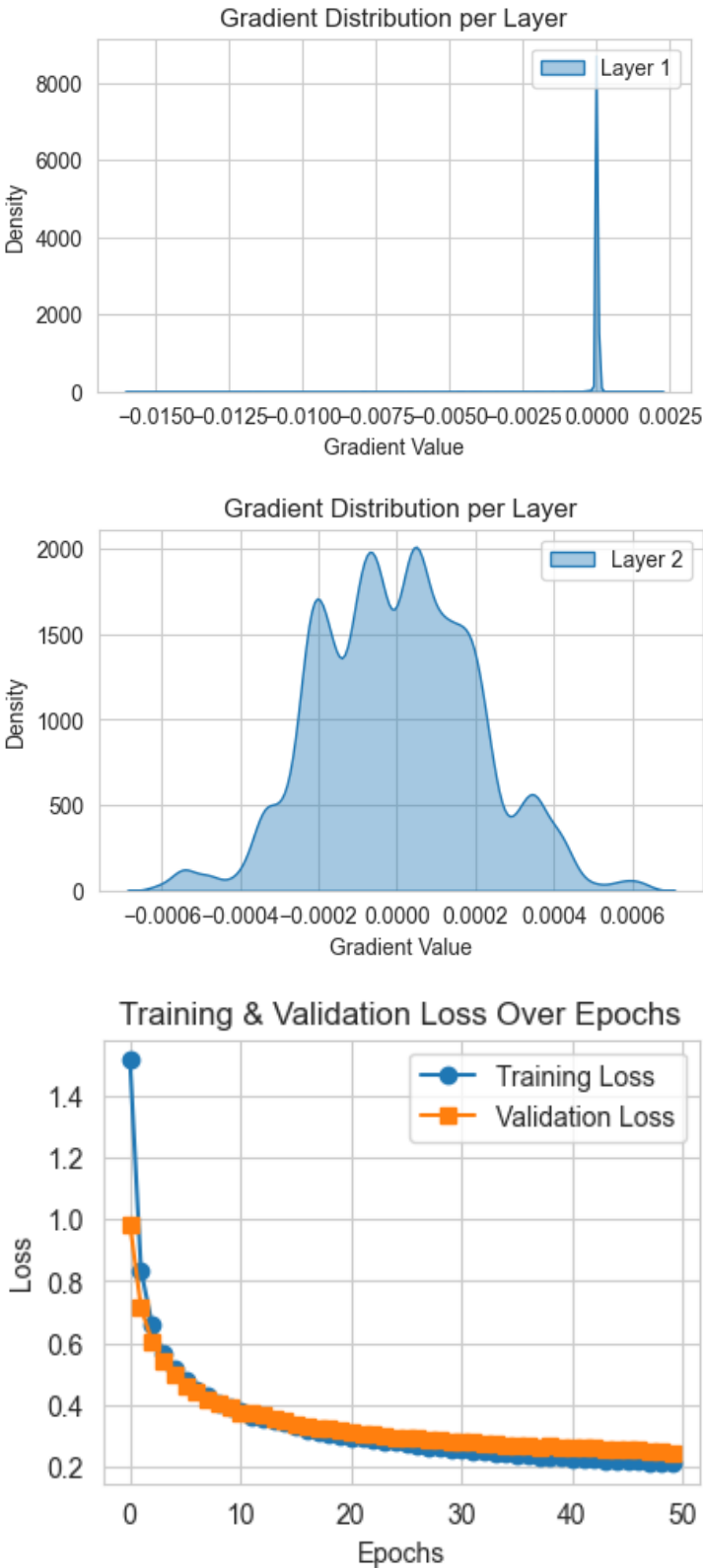
Tabel 2.2.2.1 Hasil Pengujian terhadap fungsi aktivasi pada hidden layer

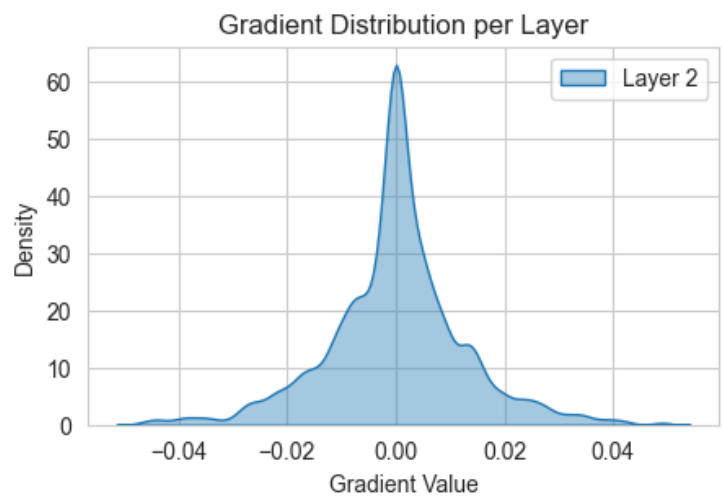
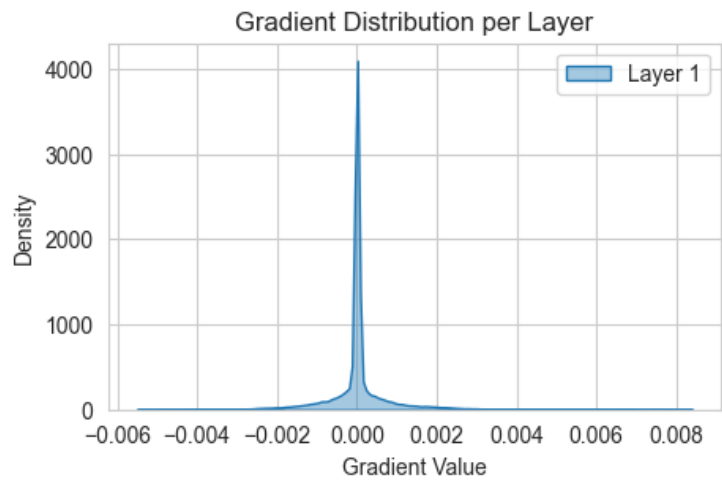
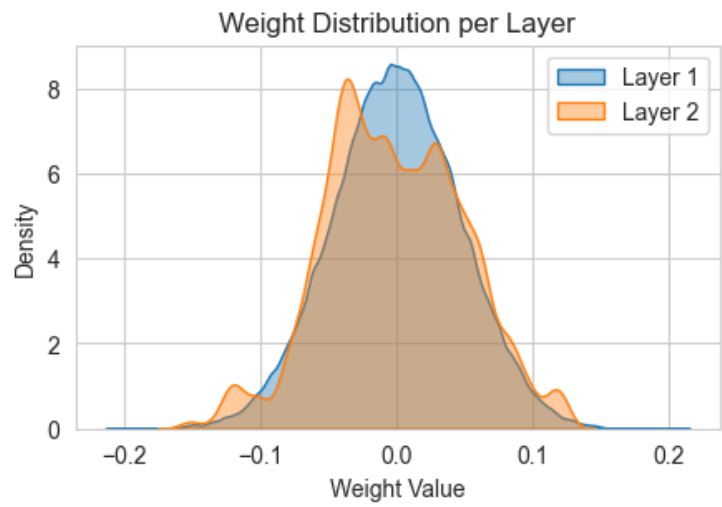
Fungsi aktivasi	Hasil
relu	Val accuracy: 92.91%, Test accuracy: 94.53%

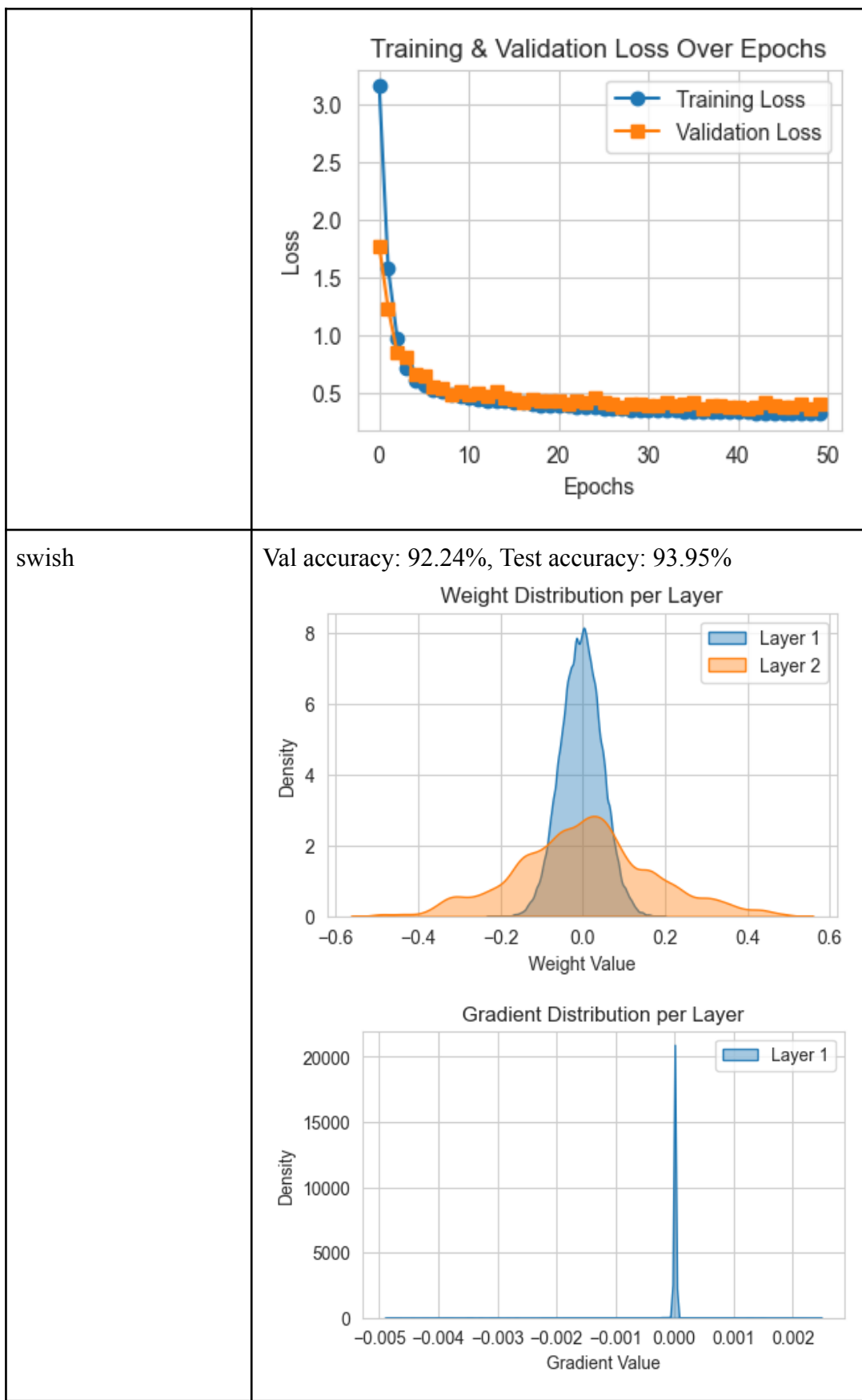


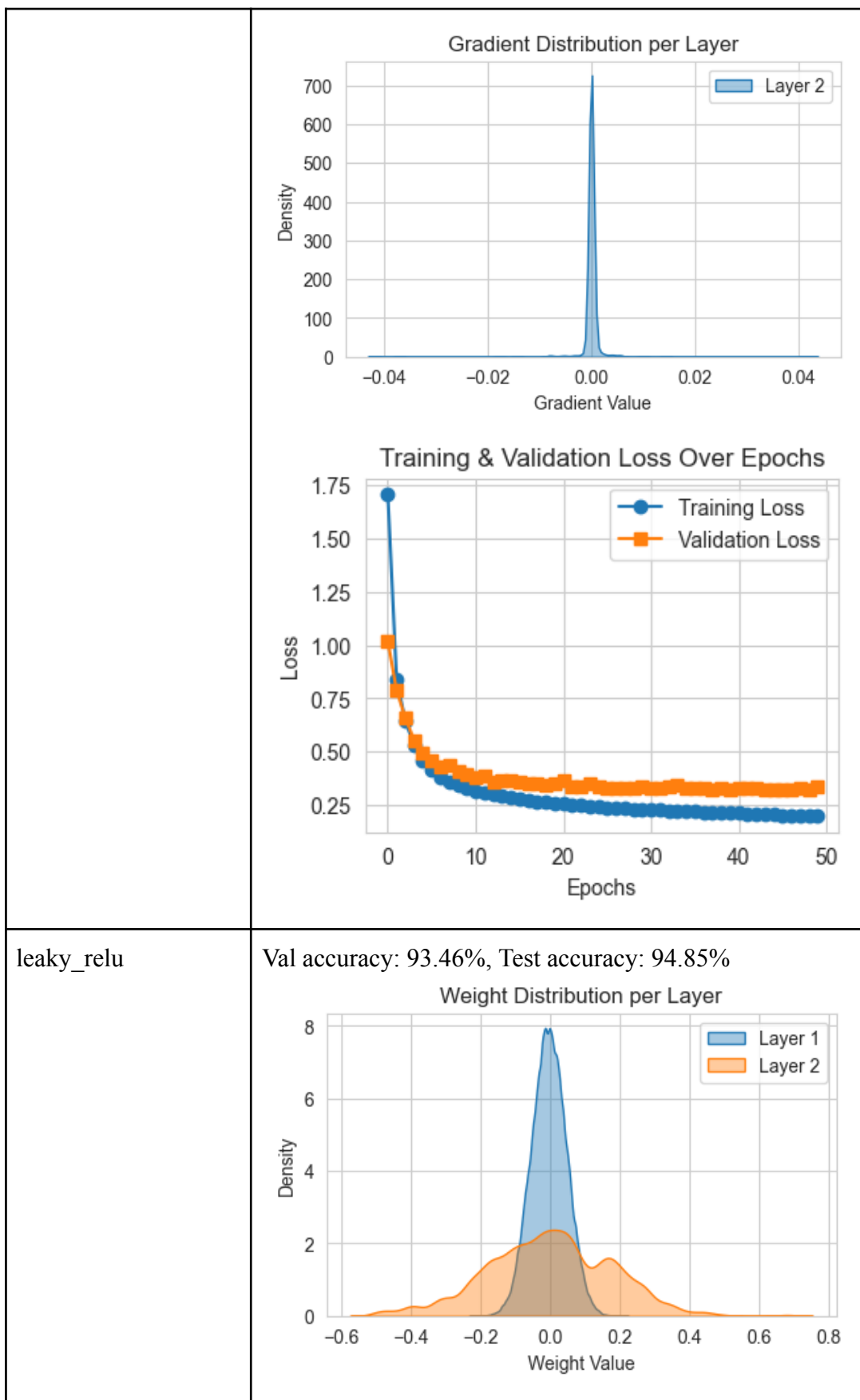


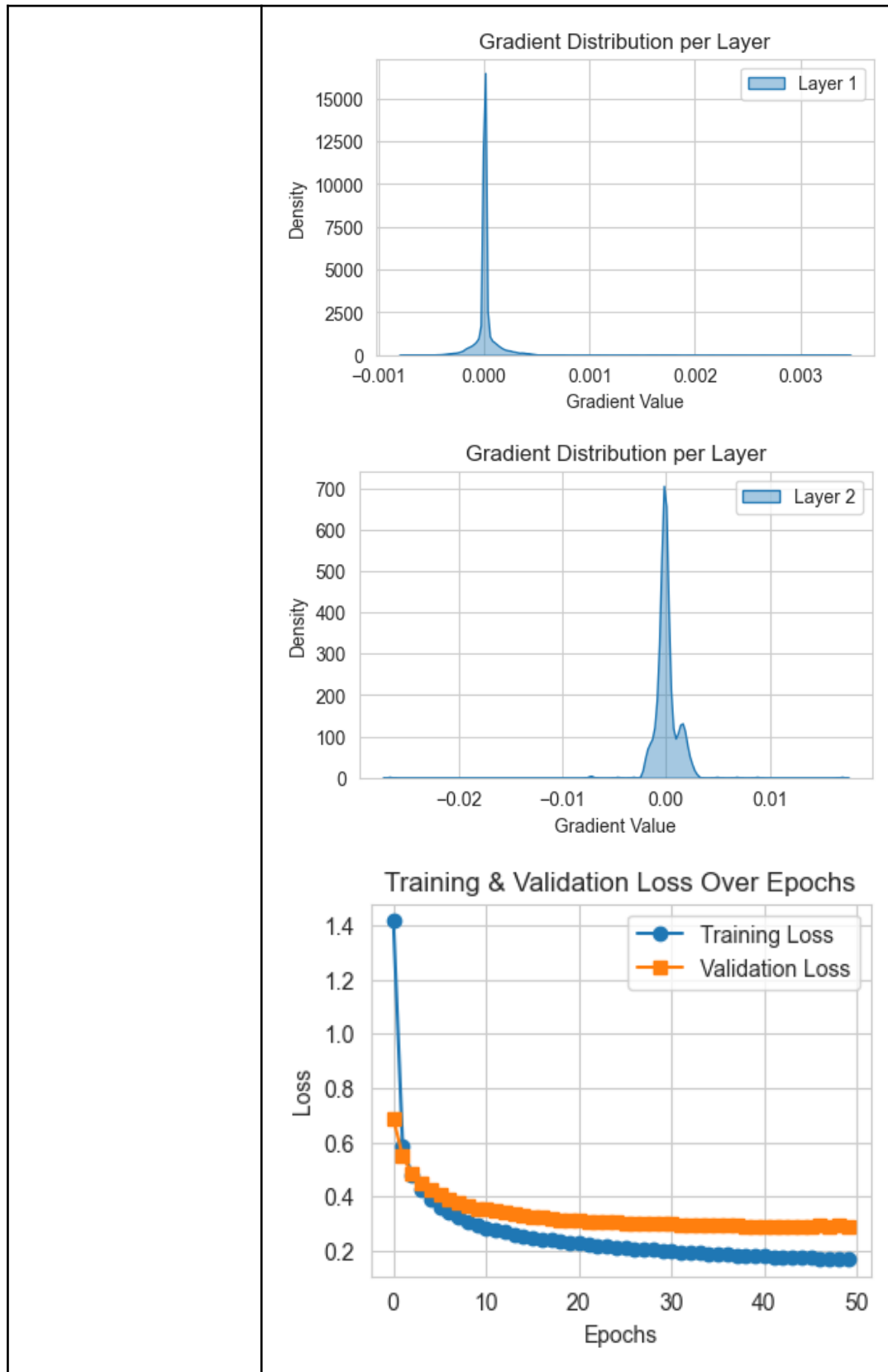


	 The figure consists of three subplots: Gradient Distribution per Layer (Layer 1): A density plot showing a sharp peak at 0.0000, indicating that gradients for Layer 1 are concentrated near zero. Gradient Distribution per Layer (Layer 2): A density plot showing a broad distribution with multiple peaks, primarily centered around 0.0000, indicating more varied gradient values for Layer 2. Training & Validation Loss Over Epochs: A line plot showing both training and validation loss decreasing over 50 epochs. The training loss (blue line with circles) starts at approximately 1.5 and ends near 0.2. The validation loss (orange line with squares) starts at approximately 1.0 and ends near 0.25.
linear	Val accuracy: 89.58%, Test accuracy: 91.14%









a. Perbandingan hasil akhir prediksi

Berdasarkan hasil pada tabel, terlihat bahwa daftar fungsi aktivasi dengan kinerja hasil akhir prediksi mulai dari yang terbaik adalah leaky_relu, relu, tanh, swish, sigmoid, dan linear. Fungsi aktivasi linear mungkin memiliki kinerja paling rendah karena seakan-akan tidak menggunakan fungsi aktivasi sehingga sulit untuk mencari pola nonlinier. Hasil yang didapatkan juga mungkin karena inisialisasi bobot yang dipakai adalah He, sedangkan He cocok digunakan bersama ReLU atau Leaky ReLU.

b. Perbandingan grafik loss pelatihan

Berdasarkan grafik loss pelatihan, dapat dilihat bahwa secara rata-rata, semua model memiliki nilai loss sekitar 0.2 - 0.3 pada akhir epoch. Akan tetapi, model dengan fungsi aktivasi linear memiliki nilai loss yang mendekati 0.5. Model tersebut juga mengalami plateau pada epoch yang lebih cepat dibandingkan dengan model lainnya.

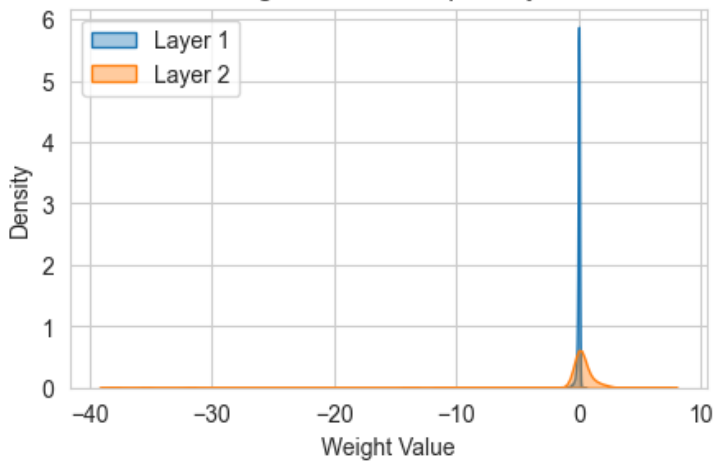
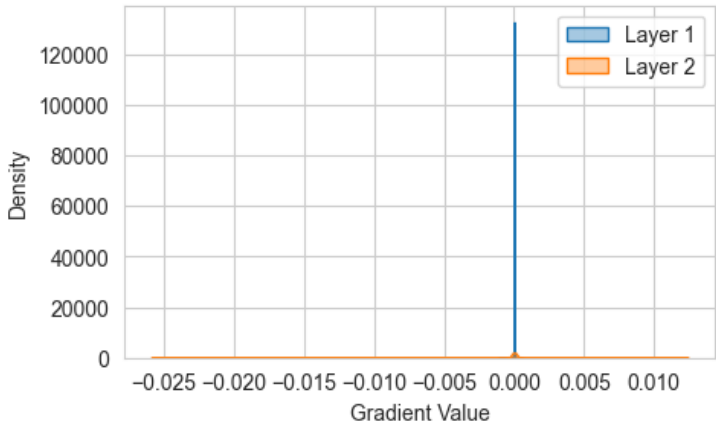
c. Perbandingan distribusi bobot dan gradien bobot

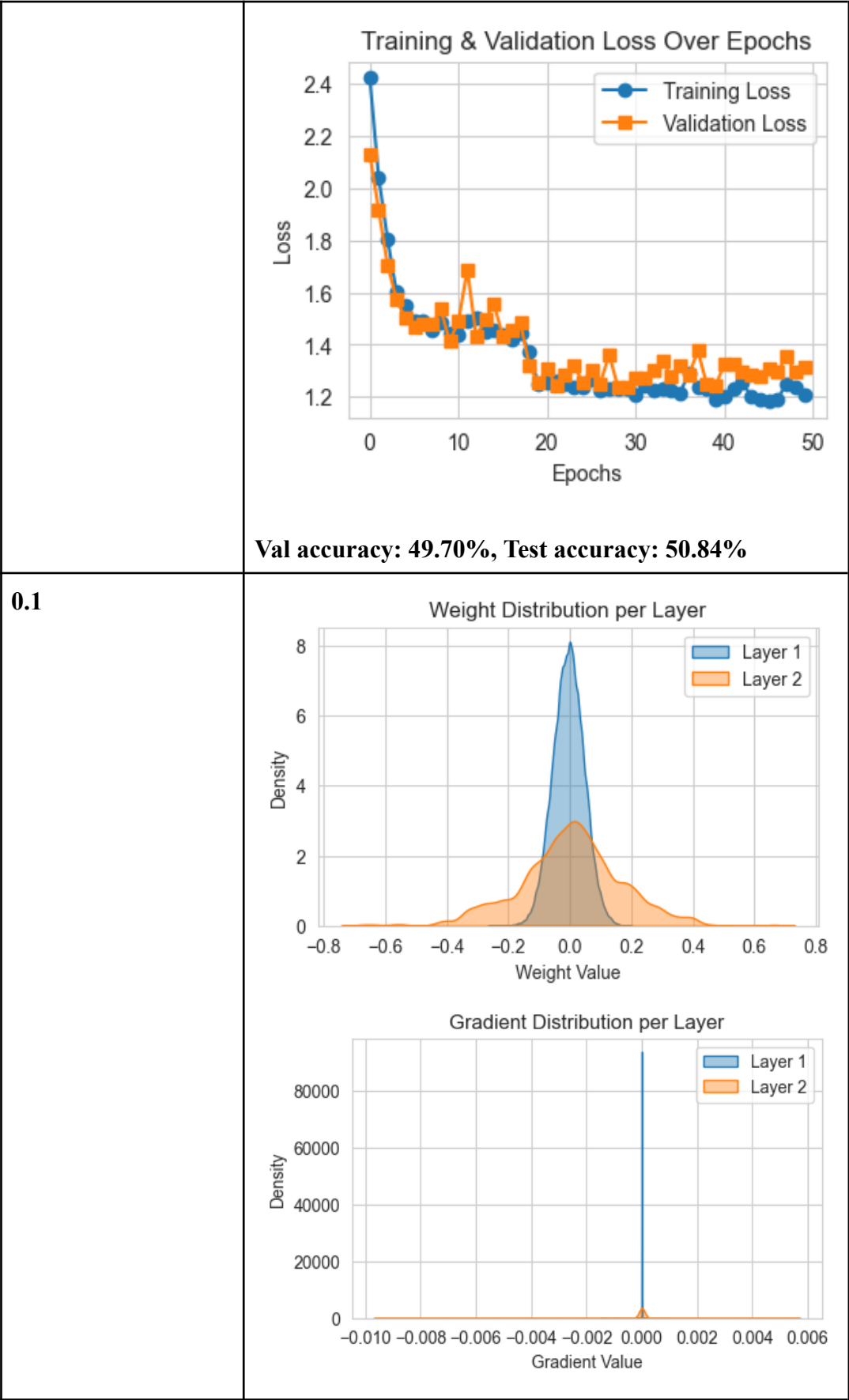
Berdasarkan grafik distribusi bobot, terlihat bahwa bobot pada layer satu memiliki distribusi bobot yang mirip untuk setiap fungsi aktivasi, yaitu bobot sekitar -0.2 dan 0.2. Akan tetapi, pada layer kedua, bentuk distribusi bobot untuk setiap fungsi aktivasi lebih bervariasi, namun memiliki nilai di antara -0.6 dan 0.6 kecuali untuk model dengan fungsi aktivasi linear (bobot antara -0.2 dan 0.2).

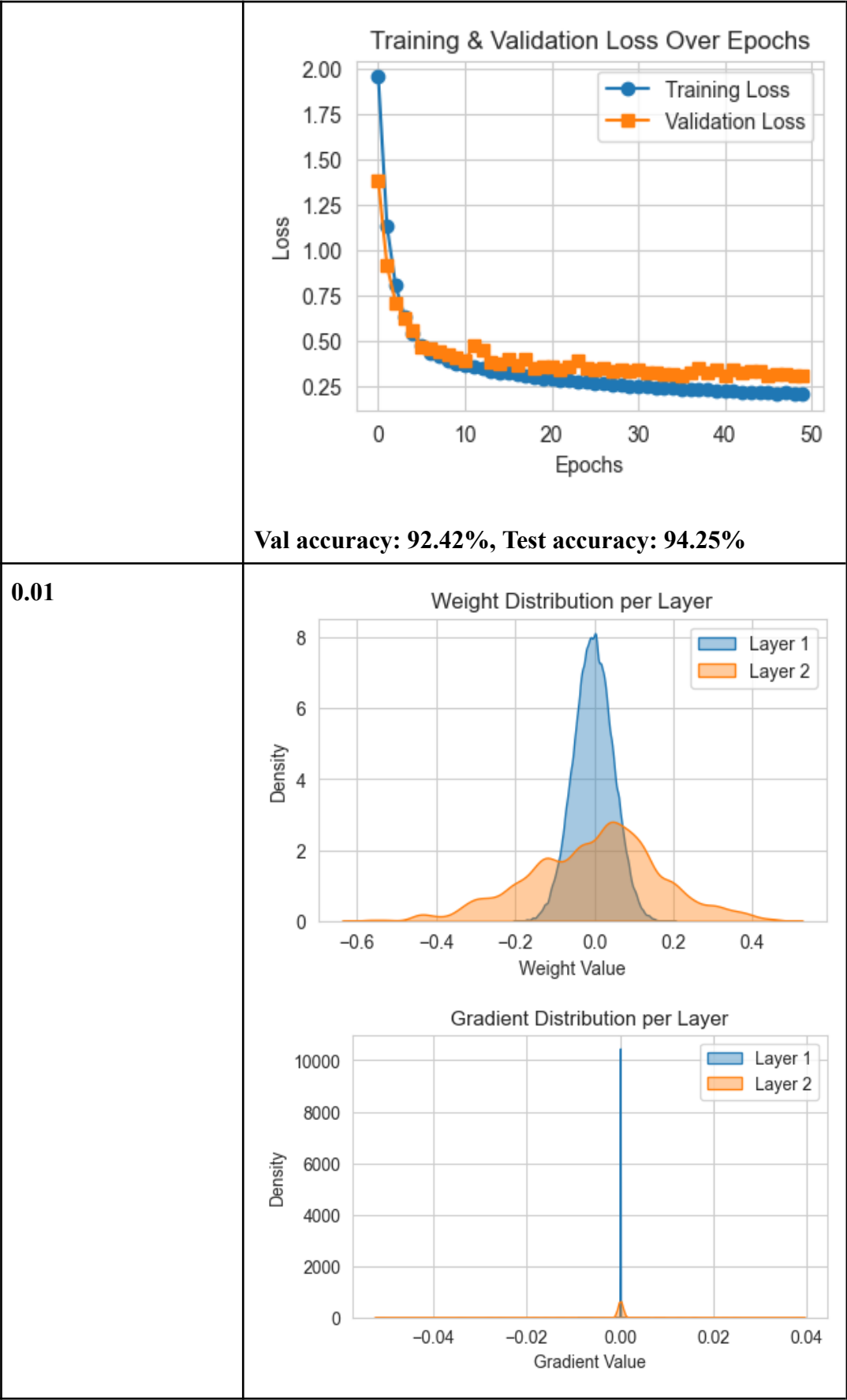
Berdasarkan grafik distribusi gradien bobot, terlihat bahwa gradien pada layer pertama memiliki pola yang serupa pada semua fungsi aktivasi, yaitu mendekati nol. Pada layer kedua, terlihat bahwa fungsi aktivasi dengan distribusi gradien bobot terbesar adalah fungsi linear. Fungsi sigmoid dan tanh memiliki distribusi gradien bobot di antara -0.0006 hingga 0.0006. Fungsi ReLU, swish, dan Leaky ReLU menghasilkan distribusi yang mayoritas berada di rentang -0.002 hingga 0.002, tetapi memiliki banyak outlier di luar rentang tersebut.

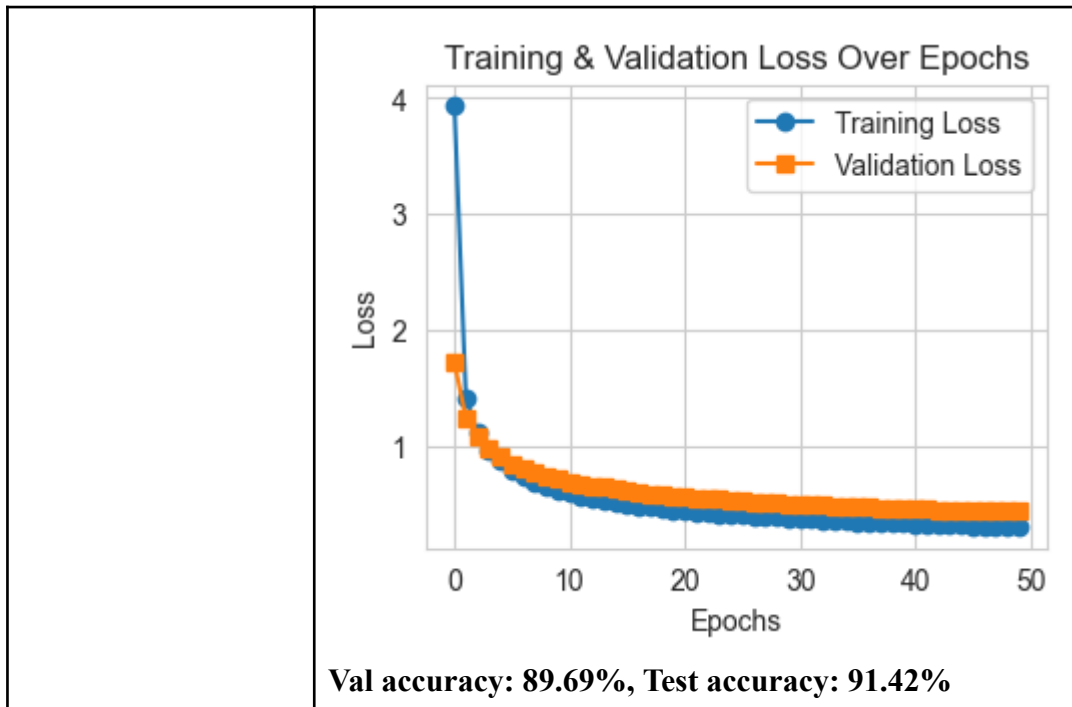
2.2.3. Pengaruh learning rate

Tabel 2.2.3.1 Hasil Pengujian terhadap learning rate

Learning rate	Hasil
0.5	Weight Distribution per Layer  Gradient Distribution per Layer 







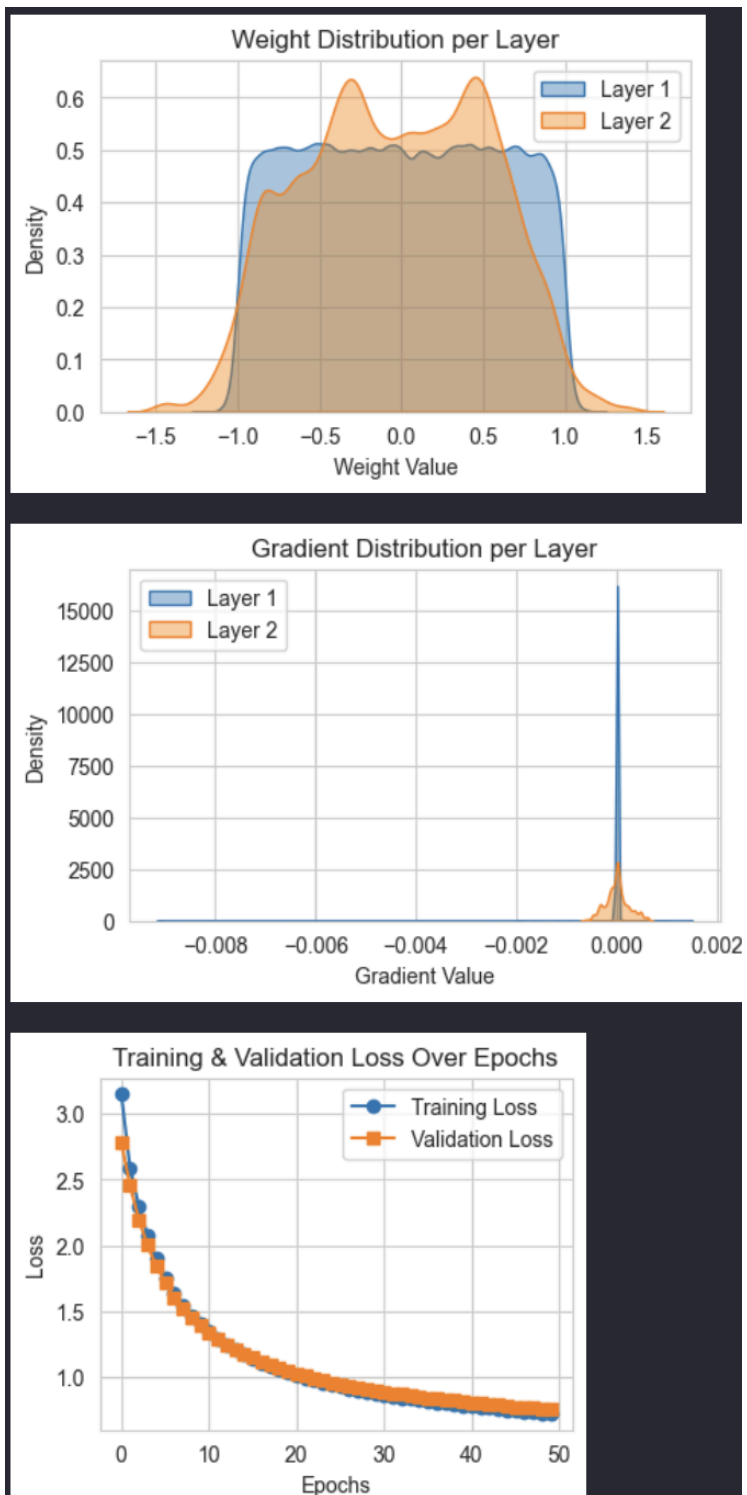
Learning rate 0.5 menghasilkan akurasi yang paling kecil. Hal ini menunjukkan bahwa learning rate yang terlalu besar menyebabkan sulitnya model untuk konvergen dan menemukan pattern yang optimal. Penurunan training dan validation loss juga cenderung mengalami fluktuasi pada learning rate ini. Pada learning rate 0.1, model menunjukkan performa dan akurasi terbaik. Training loss dan validation loss menurun dengan pola lebih stabil, walaupun masih ada sedikit fluktuasi di epoch ke-10 untuk validation loss. Untuk learning rate 0.01, akurasi masih tergolong baik tetapi sedikit lebih rendah jika dibandingkan dengan learning rate 0.1. Hal ini mengindikasikan bahwa model tidak cukup belajar dari dataset yang ada karena pembaruan bobot yang terlalu kecil di tiap iterasi. Dengan learning rate sekecil ini, gradien yang diperbarui di setiap iterasi menjadi sangat kecil sehingga model tidak dapat menyesuaikan bobotnya secara efektif. Akibatnya, model mengalami underfitting, di mana model gagal menangkap pola yang cukup dari data train untuk mencapai performa yang optimal.

2.2.4. Pengaruh inisialisasi bobot

Tabel 2.2.4.1 Hasil Pengujian terhadap inisialisasi bobot

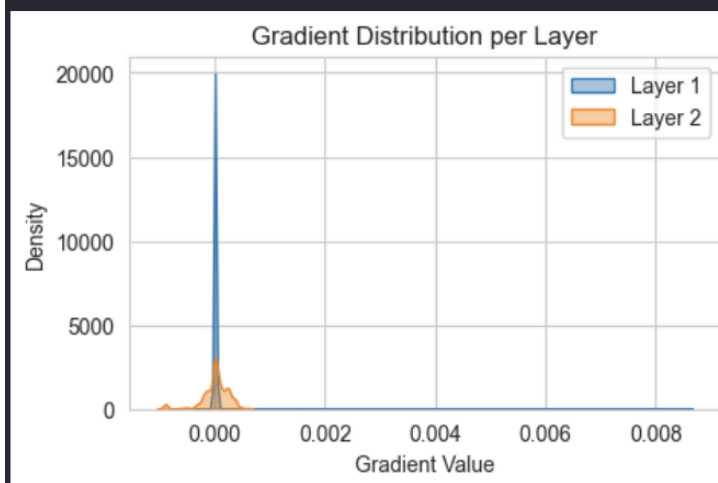
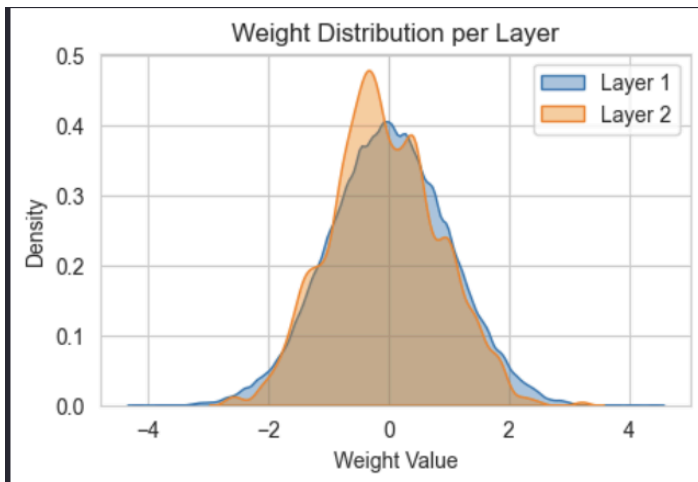
inisialisasi	Hasil
zero	Weight Distribution per Layer This plot shows the density of weight values for Layer 1 (blue) and Layer 2 (orange). The x-axis represents the Weight Value from -0.10 to 0.10, and the y-axis represents the Density from 0 to 600. Both layers show a sharp peak at 0.00, indicating that the weights are initialized to zero. Gradient Distribution per Layer This plot shows the density of gradient values for Layer 1 (blue) and Layer 2 (orange). The x-axis represents the Gradient Value from -0.003 to 0.000, and the y-axis represents the Density from 0 to 7000. Both layers show a sharp peak at 0.000, indicating that the gradients are initialized to zero. Training & Validation Loss Over Epochs This plot shows the training and validation loss over 50 epochs. The x-axis represents the Epochs from 0 to 50, and the y-axis represents the Loss from 1.80 to 2.10. The Training Loss (blue line with circles) and Validation Loss (orange line with squares) both decrease over time, starting around 2.05 and ending around 1.82. Val accuracy: 23.26%, Test accuracy: 23.25%

uniform



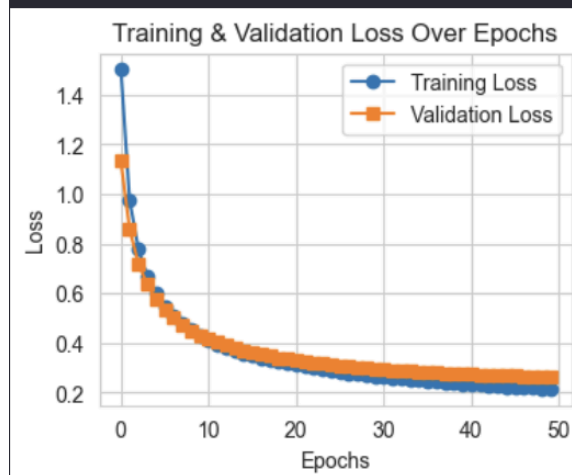
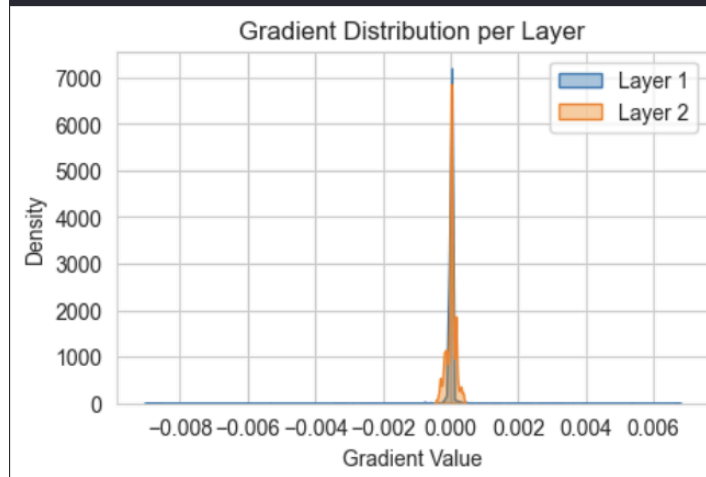
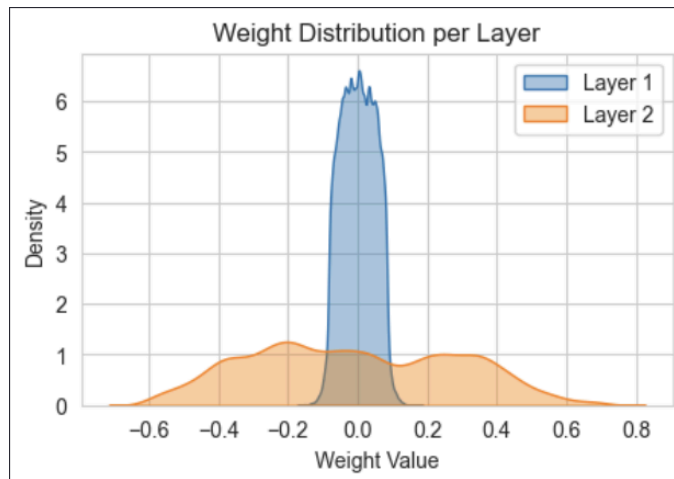
Val accuracy: 75.61%, Test accuracy: 77.48%

normal



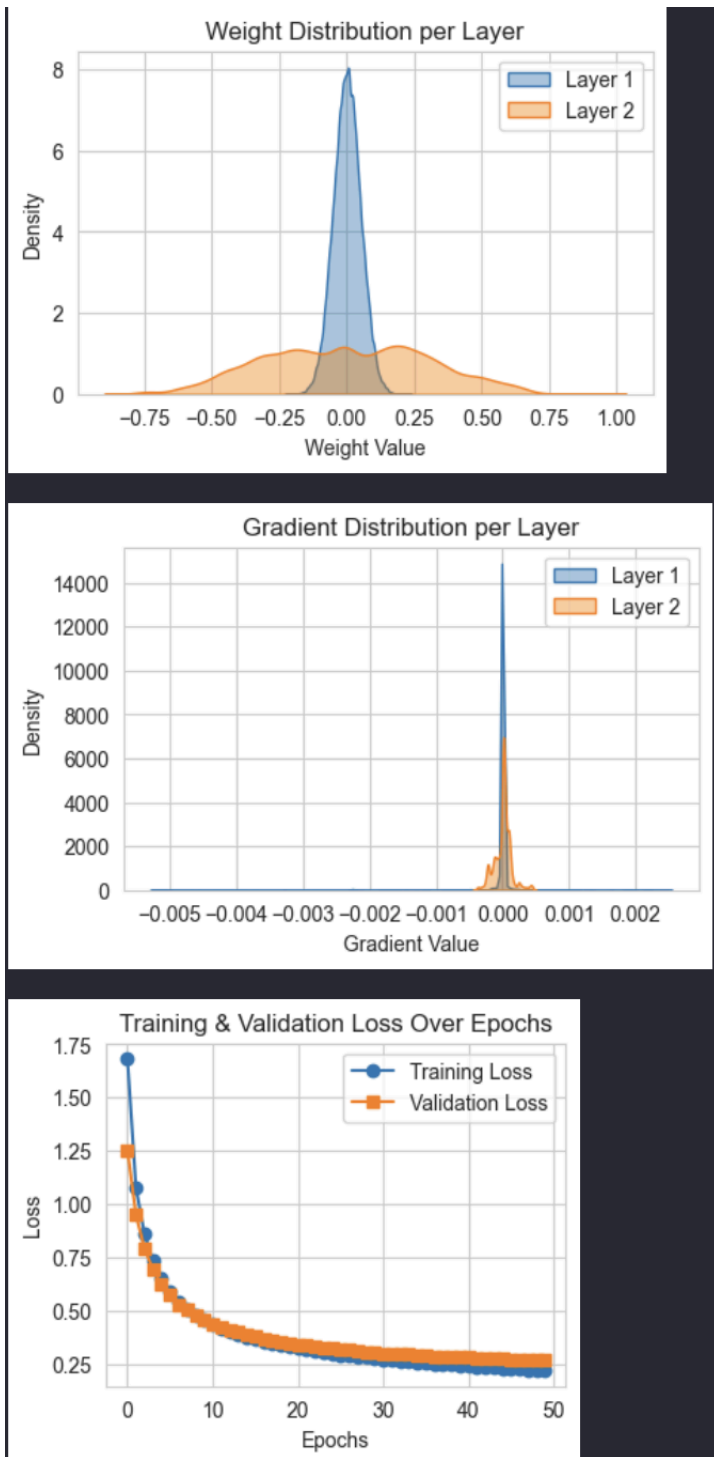
Val accuracy: 67.23%, Test accuracy: 68.58%

uniform xavier



Val accuracy: 92.80%, Test accuracy: 94.38%

normal he



Val accuracy: 92.41%, Test accuracy: 94.21%

a. Perbandingan hasil akhir prediksi

Perbandingan hasil akhir prediksi dari akurasi tertinggi hingga terendah yaitu xavier, he, uniform, normal, dan zero. Inisialisasi bobot dengan nilai 0/zero menghasilkan akurasi yang paling rendah karena adanya simetri antar

neuron dimana semua neuron dalam layer yang sama akan memiliki gradien yang identik, sehingga mereka belajar dengan cara yang sama dan kurang mampu menangkap pola yang cukup dari data. Inisialisasi bobot yang terdistribusi normal menghasilkan performa yang lebih baik, namun masih lebih rendah dari inisialisasi seragam. Inisialisasi bobot yang seragam menghasilkan model dengan performa lebih baik dibanding inisialisasi dengan nilai 0 karena bobot tiap neuron lebih bervariasi sehingga model dapat menangkap pola dari dataset. Inisialisasi He menghasilkan performa yang cukup baik karena mempertimbangkan varians yang optimal berdasarkan jumlah neuron sebelumnya. Inisialisasi bobot xavier menghasilkan model dengan akurasi terbaik dan varians yang lebih stabil sehingga mencegah terjadinya *exploding* atau *vanishing gradients*. Inisialisasi xavier dan he memberikan performa terbaik secara konsisten karena mempertimbangkan struktur jaringan untuk menghasilkan bobot awal yang seimbang.

b. Perbandingan grafik loss pelatihan

Model dengan inisialisasi bobot bernilai 0 mengalami penurunan loss yang cukup fluktuatif di pertengahan epoch. Sedangkan model dengan inisialisasi lainnya mengalami penurunan yang sedikit curam pada epoch awal dan semakin mulus pada pertengahan hingga akhir epoch.

c. Perbandingan distribusi bobot dan gradien bobot

Model dengan inisialisasi bobot 0 pada layer pertama menyebabkan bobot di setiap neuron identik. Distribusi gradiennya juga terkonsentrasi di sekitar nol. Model dengan inisialisasi uniform memiliki distribusi yang lebih merata dalam rentang -1 dan 1 sehingga gradien pada layer tersebar lebih luas dibandingkan dengan zero initialization dan menyebabkan pembelajaran yang lebih efektif. Model dengan inisialisasi normal mengikuti distribusi normal dengan mean sekitar nol, bobot tersebar dalam range yang lebih luas dibandingkan dengan zero initialization. Uniform dan normal initialization bekerja lebih baik karena memberikan bobot yang lebih tersebar dan memungkinkan propagasi gradien yang lebih baik. Inisialisasi xavier menghasilkan model dengan bobot berdasarkan jumlah

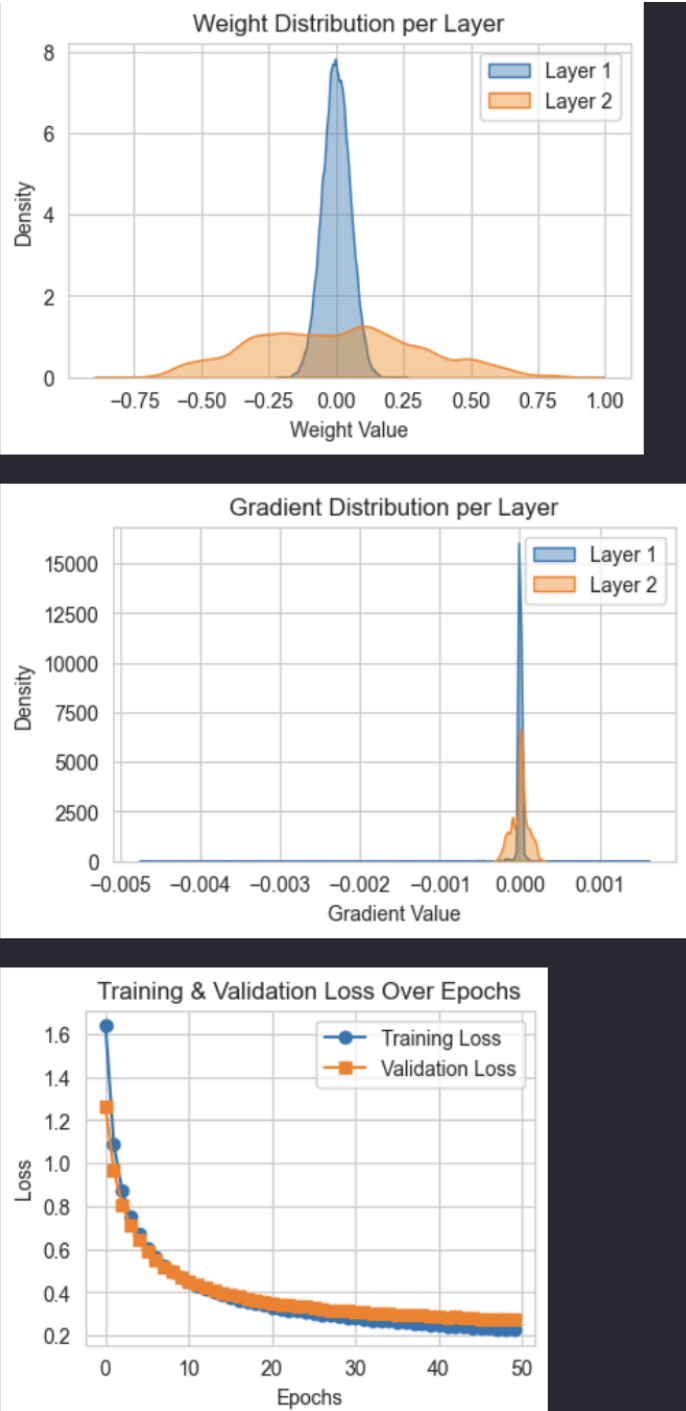
neuron input dan output, menghasilkan distribusi yang lebih stabil. Gradiennya juga lebih terdistribusi. Inisialisasi He juga mempertimbangkan nilai bobot berdasarkan jumlah neuron sebelumnya sehingga distribusi yang lebih tersebar.

2.2.5. Pengaruh regularisasi

Tabel 2.2.5.1 Hasil Pengujian terhadap regularisasi

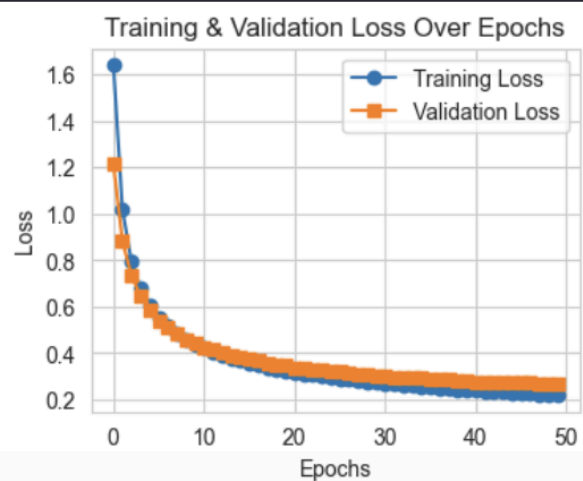
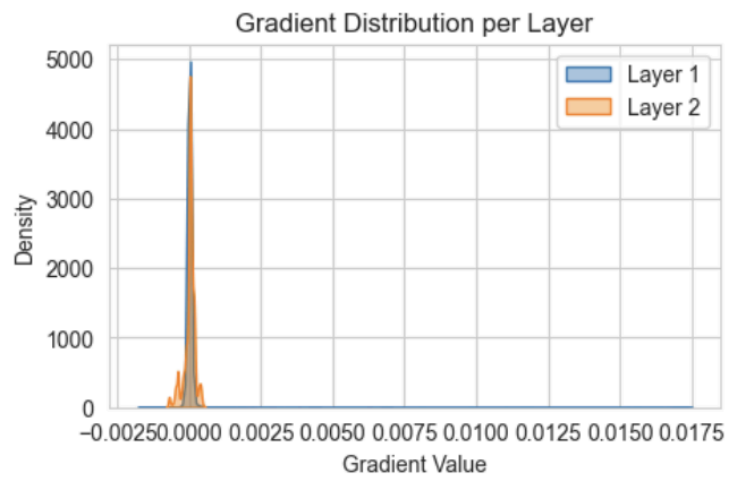
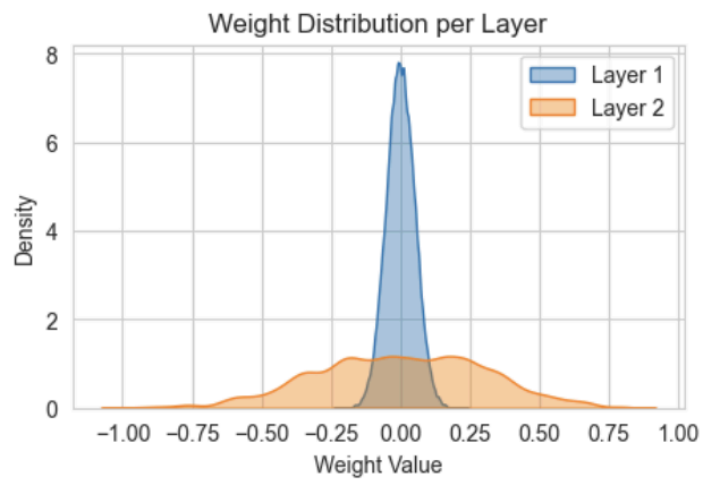
Jenis	Hasil
--------------	--------------

Tanpa regularisasi



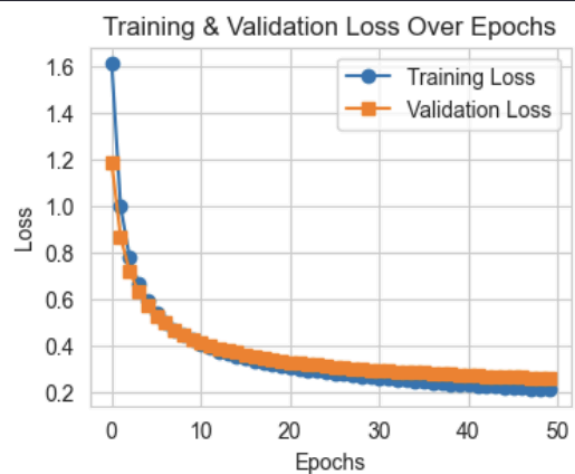
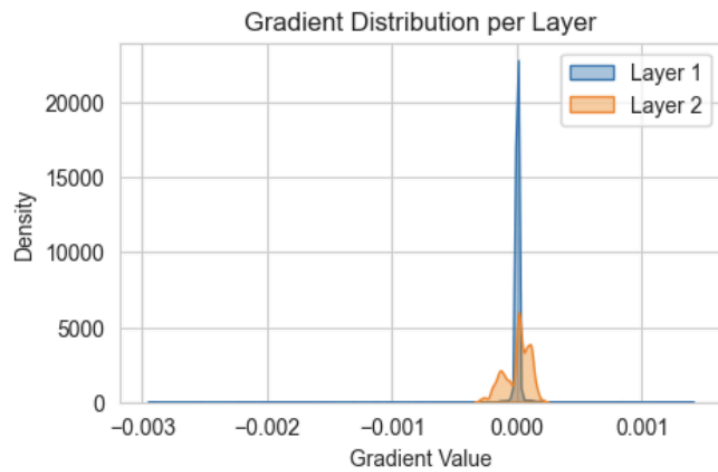
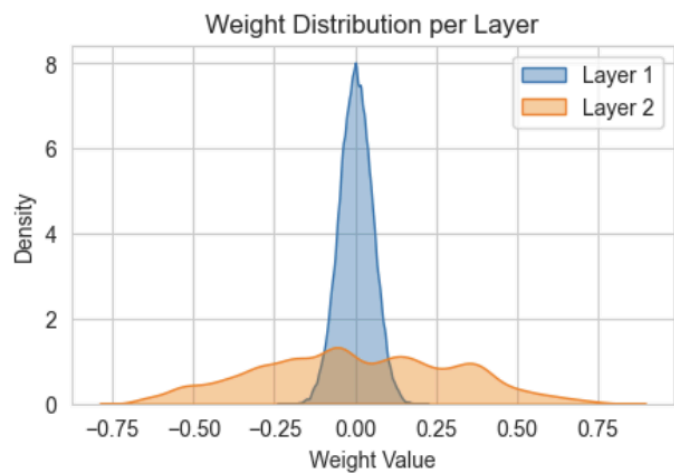
Val accuracy: 92.28%, Test accuracy: 94.25%

Regularisasi L1



Val accuracy: 92.32%, Test accuracy: 94.42%

Regularisasi L2



Val accuracy: 92.47%, Test accuracy: 94.68%

a. Perbandingan hasil akhir prediksi

Model dengan L2 memiliki validation accuracy tertinggi, hal ini menunjukkan bahwa model ini memiliki generalisasi terbaik di antara ketiganya. Model dengan regularisasi L1 menempati posisi kedua dari segi akurasi. Sementara itu, metode tanpa regularisasi memiliki performa yang paling rendah. Walaupun perbedaan akurasi antara ketiga model tidak jauh berbeda, model tanpa regularisasi lebih rentan terhadap overfitting.

b. Perbandingan grafik loss pelatihan

Semua model mengalami penurunan loss yang stabil seiring bertambahnya jumlah epoch. Model tanpa regularisasi memiliki validation loss yang lebih besar dibandingkan dengan model L1 dan L2. Hal ini menunjukkan bahwa mungkin terjadi overfitting pada model tanpa regularisasi karena model terlalu menyesuaikan diri dengan data training. Kedua model dengan regularisasi memiliki penurunan training dan validation loss yang lebih stabil, dimana model L2 lebih stabil dibanding L1 pada akhir proses training. Hal ini menunjukkan bahwa L2 lebih efektif dalam mengurangi overfitting dibandingkan model L1 dan tanpa regularisasi.

c. Perbandingan distribusi bobot dan gradien bobot

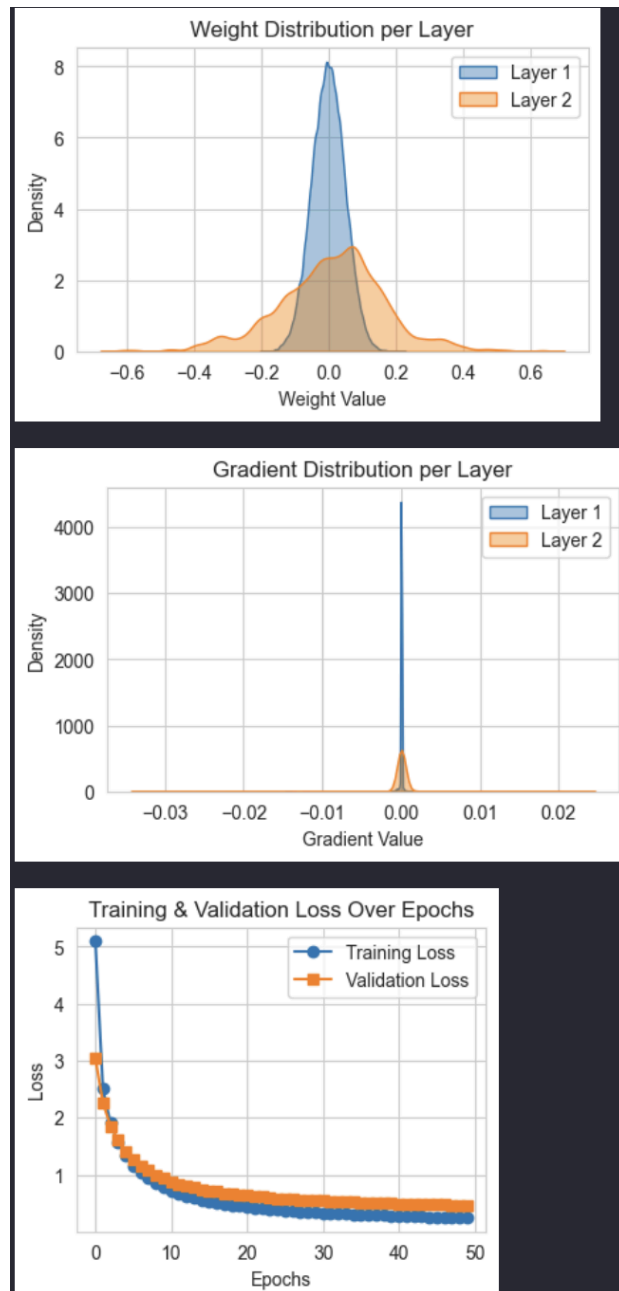
Dari segi distribusi bobot, ketiga model tidak memiliki perbedaan yang signifikan. Dari segi distribusi gradien, model tanpa regularisasi memiliki distribusi gradien yang tinggi di sekitar nol tetapi lebih tersebar untuk layer lainnya, hal ini menunjukan kemungkinan terjadinya overfitting pada model. Model dengan L1 cenderung membuat distribusi gradien lebih terpusat mendekati nol, karena banyak bobot yang ditekan ke nol.. Model dengan L2 memiliki distribusi gradien yang lebih menyebar dibandingkan dengan model L1 dan tanpa regularisasi, karena penalti kuadratnya memengaruhi bobot dengan lebih merata.

2.2.6. Pengaruh normalisasi RMSNorm

Tabel 2.2.5.1 Hasil Pengujian terhadap RMSNORM

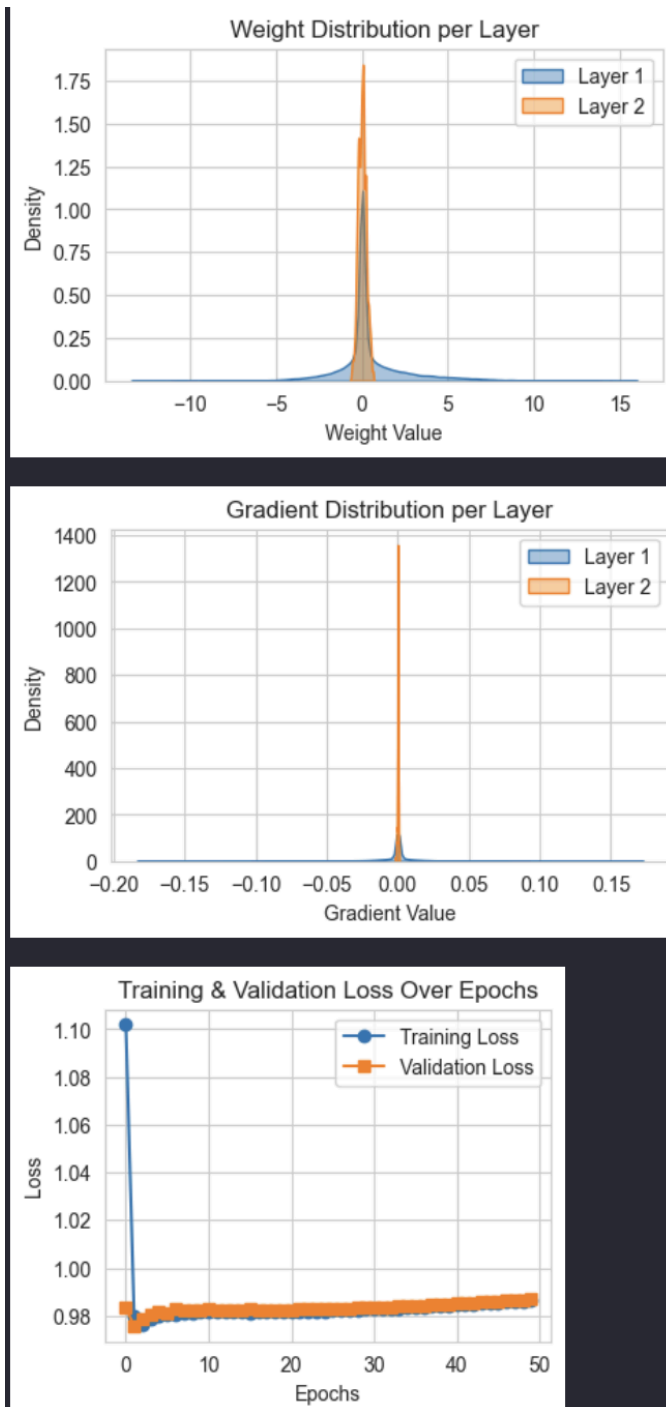
Jenis	Hasil
-------	-------

Tanpa normalisasi



Val accuracy: 90.69%, Test accuracy: 93.52%

Dengan normalisasi



Val accuracy: 83.60%, Test accuracy: 84.40%

a. Perbandingan hasil akhir prediksi

Model tanpa RMSNorm menghasilkan akurasi yang lebih baik dibandingkan model tanpa RMSNorm. Hal ini menunjukkan bahwa model tanpa RMSNorm lebih mampu melakukan generalisasi terhadap data yang

belum pernah dilihat sebelumnya. Meskipun RMSNorm dapat menormalisasi aktivasi, dalam kasus ini, proses normalisasi yang dilakukan oleh RMSNorm membuat variasi atau perbedaan antar nilai gradien menjadi terlalu kecil, sehingga model kesulitan menangkap fitur penting saat pembelajaran dan kesulitan menggeneralisasi pola data baru. Akibatnya, proses pembelajaran menjadi kurang optimal dan akurasi lebih rendah. Bisa dilihat juga bahwa model tanpa RMSNorm memiliki penurunan loss yang lebih konsisten yang juga mempengaruhi peningkatan akurasi peningkatan akurasi.

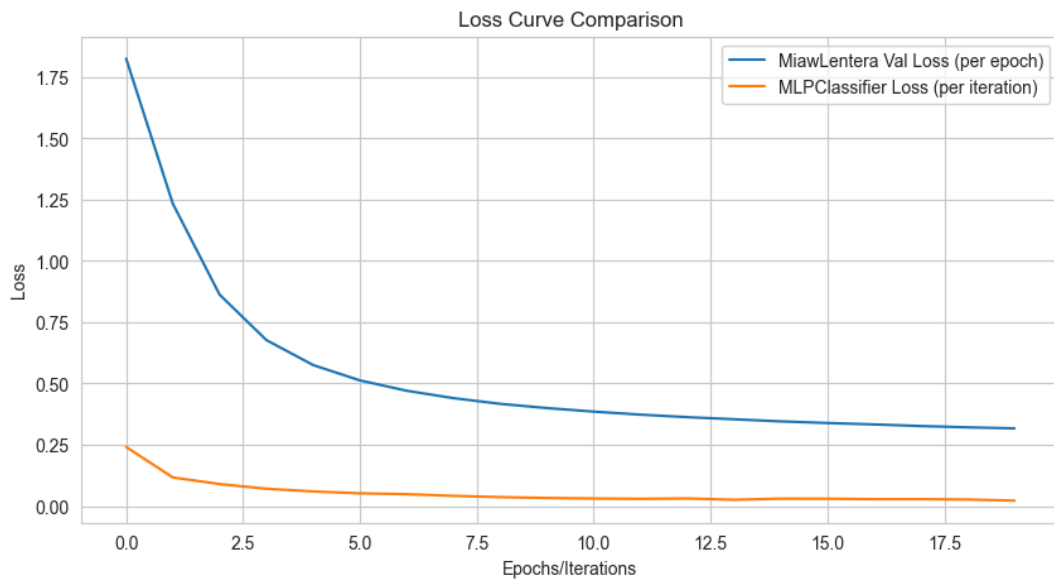
b. Perbandingan grafik loss pelatihan

Terdapat perbedaan yang cukup signifikan antara kedua model terkait grafik loss. Model dengan RMSNorm menghasilkan penurunan training loss yang signifikan di epoch pertama, dan secara keseluruhan nilai lossnya lebih rendah dibandingkan model tanpa RMSNorm. Namun, setelah penurunan awal tersebut training loss cenderung *stuck*, bahkan meningkat. Sementara itu, model tanpa RMSNorm memiliki nilai loss awal yang lebih tinggi, namun menurun dengan lebih konsisten. Perbandingan kedua grafik loss ini menunjukkan bahwa model tanpa RMSNorm lebih cepat konvergen. Model dengan RMSNorm pada kasus ini mengakibatkan *smoothing* gradien yang berlebihan sehingga update bobot menjadi lebih kecil dan kurang bervariasi, sehingga model kesulitan untuk belajar dan memperbaiki nilai lossnya.

c. Perbandingan distribusi bobot dan gradien bobot

Model dengan RMSNorm memiliki distribusi bobot yang lebih terkonsentrasi, sementara itu model tanpa RMSNorm cenderung memiliki bobot yang lebih tersebar. Hal ini dikarenakan model tanpa RMSNorm cenderung memiliki varians yang lebih tinggi yang menyebabkan ketidakstabilan bobot saat training. Distribusi gradien pada model dengan RMSNorm sangat terkonsentrasi di sekitar 0. Model tanpa RMSNorm lebih tersebar di layer kedua, walaupun masih terpusat di sekitar 0. Hal ini menunjukkan bahwa normalisasi RMSNorm menyebabkan update gradien lebih stabil dan mengurangi fluktuasi dari perubahan gradien.

2.2.7. Perbandingan dengan library sklearn



Kami melakukan perbandingan antara model dengan MLPClassifier dan implementasi dari *scratch* dengan parameter berikut

```
layer_sizes=[784, 100, 50, 10],
activations=['relu', 'relu', 'softmax'],
loss_function='cce',
weight_init='xavier',
use_rmsnorm=False,
reg_type='l2',
```

Model MLPClassifier menghasilkan akurasi sebesar 97.53%, dan model dari *scratch* menghasilkan akurasi 90.68%.

Pada model yang kami buat, loss validasi pada awal training dimulai sekitar 1.8 dan secara bertahap menurun hingga mencapai sekitar 0.32 di akhir epoch ke-20. Kurva loss menunjukkan penurunan loss yang tajam pada epoch awal, hal ini menandakan bahwa proses pembelajaran terjadi dgn cepat saat model mulai belajar dari pola-pola dasar dalam data. Namun setelah beberapa epoch, kurva mulai melandai yang menunjukkan bahwa pembelajaran terjadi lebih lambat.

Pada model dengan MLPClassifier, loss pada iterasi pertama sudah cukup rendah, sekitar 0.24, dan turun menjadi 0.05 hanya dalam 20 iterasi. Model ini bisa

mencapai hasil sangat baik dalam waktu jauh lebih singkat. Hal ini kemungkinan disebabkan oleh optimisasi internal dari MLPClassifier.

Dengan demikian, meskipun model yang kami buat belum seakurat MLPClassifier, model kami sudah menunjukkan potensi yang cukup baik dan dapat dikembangkan lebih lanjut melalui tuning parameter, seperti dapat dilihat dari performa pada bagian sebelumnya.

BAB III

Kesimpulan dan Saran

3.1. Kesimpulan

Melalui implementasi dan eksperimen Feedforward Neural Network (FFNN), hasil eksperimen menunjukkan bahwa performa model sangat dipengaruhi oleh berbagai hyperparameter, seperti arsitektur network (depth dan width), fungsi aktivasi, learning rate, metode inisialisasi bobot, penggunaan regularisasi, dan normalisasi seperti RMSNorm.

Beberapa poin penting yang dapat disimpulkan dari hasil pengujian antara lain:

- Arsitektur Jaringan: Penambahan hidden layer dan penyesuaian jumlah neuron pada hidden layer (width) secara signifikan meningkatkan performa model. Namun, penambahan depth berlebihan tanpa informasi tambahan dapat menyebabkan pembelajaran yang redundan.
- Fungsi Aktivasi: Fungsi aktivasi seperti Leaky ReLU dan ReLU memberikan hasil terbaik dalam klasifikasi, sedangkan fungsi linear menunjukkan performa yang lebih rendah karena kurang mampu mempelajari pola nonlinier.
- Learning Rate: Learning rate yang terlalu tinggi (misal 0.5) menyebabkan ketidakstabilan pembelajaran dan akurasi yang rendah, sedangkan learning rate yang terlalu rendah (misal 0.01) mengakibatkan underfitting. Learning rate 0.1 terbukti optimal dalam eksperimen ini.
- Inisialisasi Bobot: Metode inisialisasi bobot seperti Xavier dan He memberikan hasil terbaik dan stabil dalam proses training. Sebaliknya, inisialisasi zero menyebabkan simetri antar neuron dan kegagalan dalam pembelajaran.
- Regularisasi: Regularisasi L2 membantu mengurangi overfitting dan memberikan hasil validasi yang lebih stabil dibandingkan tanpa regularisasi atau L1.
- Normalisasi: Penggunaan RMSNorm pada kasus ini justru menurunkan performa model. Ini menunjukkan bahwa meskipun normalisasi bisa

menstabilisasi gradien, tapi penggunaannya perlu disesuaikan dengan konteks dataset dan struktur model.

Berdasarkan perbandingan dengan scikit-learn, model dari MLPClassifier menunjukkan akurasi yang lebih tinggi (97.53%) dibandingkan dengan model FFNN yang diimplementasikan dari awal (90.68%). Hal ini menunjukkan bahwa meskipun model dari awal sudah cukup kompeten, MLPClassifier memanfaatkan berbagai optimasi yang mempercepat konvergensi dan menghasilkan performa lebih optimal. Dari hasil visualisasi grafik loss per epoch dan distribusi bobot dan gradien, terlihat bahwa model FFNN yang dibuat dari awal mampu belajar dengan cukup baik, namun belum seefisien MLPClassifier dalam hal kecepatan dan stabilitas konvergensi.

3.2. Saran

Meskipun implementasi Feedforward Neural Network (FFNN) telah berhasil dilakukan dan menunjukkan hasil yang cukup baik, terdapat beberapa aspek yang masih dapat ditingkatkan untuk meningkatkan efisiensi, akurasi, dan generalisasi model. Oleh karena itu, beberapa saran perbaikan dan pengembangan yang dapat dilakukan di masa mendatang antara lain sebagai berikut:

- a. Peningkatan efisiensi dalam menjalankan komputasi dengan cara sistem paralelisme untuk mempercepat proses *training*.
- b. Melakukan pengujian pada dataset yang berbeda untuk menguji model
- c. Melakukan optimasi algoritma dengan menggunakan algoritma alternatif tidak hanya *gradient descent*

Lampiran

Tabel Pembagian Tugas

Nama/Nim	Tugas
13522048 Angelica Kierra Ninta Gurning	Model FFNN, Dokumen
13522050 Kayla Namira Mariadi	Model FFNN, Dokumen
13522108 Muhammad Neo Cicero Koda	Model FFNN, Dokumen

Referensi

Bendersky, E. (2016, January 19). The softmax function and its derivative. Eli Bendersky's website. <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>

scikit-learn developers. (n.d.). MLPClassifier. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Medium. (2020, April 12). Deep dive into deep learning layers: RMSNorm and batch normalization. Medium. <https://2020machinelearning.medium.com/deep-dive-into-deep-learning-layers-rmsnorm-and-batch-normalization-b2423552be9f>

Medium. (2018, June 8). L1 and L2 regularization methods. Medium. <https://medium.com/data-science/l1-and-l2-regularization-methods-ce25e7fc831c>

Kashyap, P. (2023, September 5). Mastering weight initialization in neural networks: A beginner's guide. Medium. <https://medium.com/@piyushkashyap045/mastering-weight-initialization-in-neural-networks-a-beginners-guide-6066403140e9>

Osajima, J. (n.d.). Forward propagation in neural networks. Jason Osajima's website. <https://www.jasonosajima.com/forwardprop>

Osajima, J. (n.d.). Backpropagation in neural networks. Jason Osajima's website. <https://www.jasonosajima.com/backprop>

NumPy Developers. (n.d.). NumPy documentation (Version 2.2). NumPy. <https://numpy.org/doc/2.2/>

OpenStax. (n.d.). The chain rule for multivariable functions. LibreTexts. [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions)

Orr, D. (2021, November 11). Automatic differentiation: How do computers compute derivatives? Douglas Orr's website. <https://douglassorr.github.io/2021-11-autodiff/article.html>